

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2024 р.

Дипломний проєкт

на здобуття ступеня бакалавра

за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»

спеціальності «121 Інженерія програмного забезпечення»

на тему: Геолокаційна гра з використанням розширеної реальності

Виконав студент IV курсу, групи ІП-01
(шифр групи)

Берлінський Ярослав Владленович _____
(прізвище, ім'я, по батькові) (підпис)

Керівник асистент Храмченко М. С. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент доцент, к.т.н., доцент каф. ІСТ Сперкач М.С. _____
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2024

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ____ ” _____ 2024 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Берлінський Ярослав Владленович
(прізвище, ім'я, по батькові)

1. Тема проєкту Геолокаційна гра з використанням розширеної реальності
керівник проєкту Храмченко Микола Сергійович, асистент
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «27» травня 2024 р. №2113-с

2. Термін подання студентом проєкту « 17 » червня 2024 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроєктне обстеження предметної області: аналіз предметної області, аналіз існуючих рішень, опис бізнес-процесів, постановка задачі.

2) Розроблення вимог до програмного забезпечення: варіанти використання програмного забезпечення, аналіз системних вимог, розроблення функціональних вимог, розроблення нефункціональних вимог

3) Конструювання та розроблення програмного забезпечення: архітектура програмного забезпечення, обґрунтування вибору засобів розробки, конструювання програмного забезпечення, аналіз безпеки даних.

4) Аналіз якості та тестування програмного забезпечення: аналіз якості ПЗ, опис процесів тестування, опис контрольного прикладу.

5) Розгортання та супровід програмного забезпечення: розгортання програмного забезпечення, супровід програмного забезпечення.

5. Перелік графічного матеріалу

- 1) Схема структурна варіантів використання _____
- 2) Схема структурна класів програмного забезпечення _____
- 3) Схема бази даних _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «11» березня 2024 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	11.03.2024	
2	Аналіз існуючих методів розв'язання задачі	21.01.2024	
3	Постановка та формалізація задачі	03.03.2024	
4	Розробка інформаційного забезпечення	19.04.2024	
5	Алгоритмізація задачі	30.04.2024	
6	Обґрунтування вибору використаних технічних засобів	30.04.2024	
7	Розробка програмного забезпечення	05.05.2024	
8	Налагодження програми	10.05.2024	
9	Виконання графічних документів	20.05.2024	
10	Оформлення пояснювальної записки	20.05.2024	
11	Подання ДП на попередній захист	02.06.2024	
12	Подання ДП рецензенту	10.06.2024	
13	Подання ДП на основний захист	14.06.2024	

Студент _____
(підпис)

Ярослав БЕРЛІНСЬКИЙ
(ініціали, прізвище)

Керівник _____
(підпис)

Микола ХРАМЧЕНКО
(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 43 таблиці, 22 рисунка та 25 джерел – загалом 77 сторінок.

Дипломний проєкт присвячений розробці геолокаційної гри з використанням розширеної реальності на платформі iOS, що дозволяє користувачам взаємодіяти з віртуальними об'єктами в реальному світі та отримувати нагороди в рамках програм лояльності.

Метою роботи є створення функціонального прототипу, який продемонструє можливості інтеграції геолокаційних та AR технологій у мобільні додатки, а також підвищить залученість користувачів і запропонує нові форми взаємодії.

У першому розділі проведено обстеження предметної області, здійснено аналіз поточного стану технологій геолокації та розширеної реальності, а також існуючих рішень, що використовуються у мобільних іграх, описано бізнес-процеси та сформульовано основні задачі проєкту.

У другому розділі розроблено вимоги до програмного забезпечення, включаючи аналіз варіантів його використання, системні вимоги, а також функціональні та нефункціональні вимоги.

У третьому розділі спроектовано архітектуру програмного забезпечення, обґрунтовано вибір засобів розробки, розроблено програмне забезпечення.

У четвертому розділі проаналізовано якість програмного забезпечення, описано процеси тестування.

У п'ятому розділі описано процес розгортання програмного забезпечення та його супроводу.

КЛЮЧОВІ СЛОВА: МОБІЛЬНИЙ ЗАСТОСУНОК, РОЗШИРЕНА РЕАЛЬНІСТЬ, ГЕОЛОКАЦІЯ, IOS, XCODE, SWIFTUI, VAPOR, БАЗА ДАНИХ.

ABSTRACT

The explanatory note of the diploma project consists of five sections, containing 43 tables, 22 figures, and 25 sources – totaling 77 pages. The diploma project is dedicated to the development of a geolocation game using augmented reality on the iOS platform, allowing users to interact with virtual objects in the real world and receive rewards within loyalty programs.

The aim of the work is to create a functional prototype that demonstrates the possibilities of integrating geolocation and AR technologies into mobile applications, enhances user engagement, and offers new forms of interaction.

In the first section, a survey of the subject area was carried out, an analysis of the current state of geolocation and augmented reality technologies, as well as existing solutions used in mobile games was carried out, business processes were described and the main tasks of the project were formulated.

The second section develops the requirements for the software, including an analysis of its usage scenarios, system requirements, as well as functional and non-functional requirements.

The third section designs the software architecture, justifies the choice of development tools, and develops the software.

In the fourth section, the quality of the software is analyzed, and the testing processes are described.

The fifth section describes the process of deploying and maintaining the software.

KEYWORDS: MOBILE APPLICATION, AUGMENTED REALITY, GEOLOCATION, IOS, XCODE, SWIFTUI, VAPOR, DATABASE.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард Жаріков

“__” _____ 2024 р.

ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ РЕАЛЬНОСТІ

Технічне завдання

КП.ІП-0104.045490.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав БЕРЛІНСЬКИЙ

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2	ПІДСТАВА ДЛЯ РОЗРОБКИ.....	4
3	ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1	Вимоги до функціональних характеристик	6
4.1.1	Користувацького інтерфейсу.....	6
4.1.2	Для користувача:.....	6
4.1.3	Для адміністратора системи:	7
4.1.4	Додаткові вимоги:.....	7
4.2	Вимоги до надійності	8
4.3	Умови експлуатації.....	8
4.3.1	Вид обслуговування	8
4.3.2	Обслуговуючий персонал	8
4.4	Вимоги до складу і параметрів технічних засобів	8
4.5	Вимоги до інформаційної та програмної сумісності	8
4.5.1	Вимоги до вхідних даних.....	8
4.5.2	Вимоги до вихідних даних.....	8
4.5.3	Вимоги до мови розробки	9
4.5.4	Вимоги до середовища розробки	9
4.5.5	Вимоги до представленню вихідних кодів	9
4.6	Вимоги до маркування та пакування.....	9
4.7	Вимоги до транспортування та зберігання.....	9
4.8	Спеціальні вимоги.....	9
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	10
5.1	Попередній склад програмної документації	10
5.2	Спеціальні вимоги до програмної документації.....	10
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ	11
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	12

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ РЕАЛЬНОСТІ

Галузь застосування: Розважальні та маркетингові програми лояльності

Наведене технічне завдання поширюється на розробку мобільного додатку, що використовує технології геолокації та розширеної реальності для інтерактивних ігор. Додаток призначений для створення взаємодій з віртуальними об'єктами у реальному світі, отримуючи за це нагороди та бонуси. Цей додаток орієнтований на компанії, що бажають залучити клієнтів через інноваційні маркетингові кампанії, створюючи інтерактивні програми лояльності. Сфери застосування включають розважальні заходи, туристичні проекти, освітні програми та рекламні кампанії, де інтеграція віртуального контенту може підвищити інтерес і залучення аудиторії.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки геолокаційної гри з використанням розширеної реальності є завдання на дипломне проектування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для створення інтерактивного мобільного додатку, що поєднує технології геолокації та розширеної реальності з метою підвищення залученості користувачів у маркетингові програми лояльності та розважальні проекти.

Метою розробки є інноваційний засіб взаємодії між програмами лояльності та споживачами, який покращує користувацький досвід через інтеграцію розширеної реальності.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацького інтерфейсу

- ПЗ має відповідати стандартам Human Interface Guidelines
- Відображення екрану входу (рис. 4.1а)
- Відображення екрану перегляду завдань (рис. 4.1б)
- Відображення екрану перегляду інтерактивної мапи (рис. 4.1в)
- Відображення екрану взаємодії з віртуальними об'єктами (рис. 4.1г)
- Відображення екрану використання нагород (рис. 4.1д)

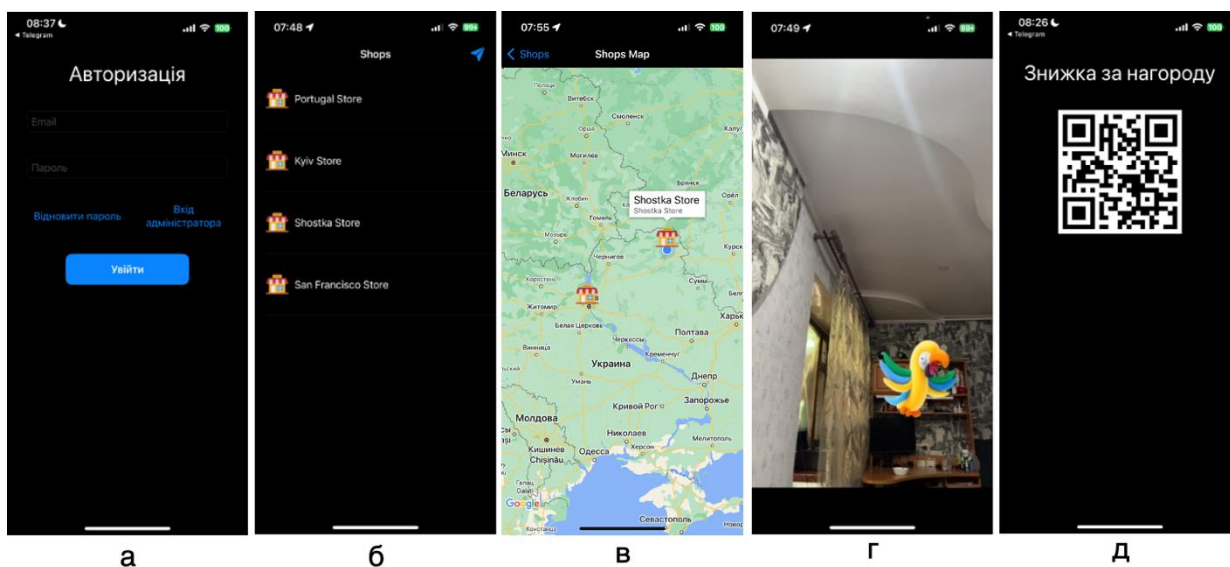


Рисунок 4.1 – UX дизайн основних екранів проекту

4.1.2 Для користувача:

- Можливість створення нового облікового запису користувача з введенням необхідних даних, таких як ім'я, email та пароль
- Можливість входу в систему з використанням електронної пошти та паролю

- Можливість зміни паролю за допомогою введення поточного паролю та нового паролю
- Можливість введення електронної пошти для отримання коду підтвердження
- Можливість введення коду підтвердження для завершення процесу зміни паролю
- Можливість почати гру, що використовує геолокацію та розширену реальність.
- Можливість визначення поточного місцезнаходження користувача
- Можливість відображення цілей гри на карті, інтегрований у додаток
- Можливість взаємодії з віртуальними об'єктами через камеру пристрою
- Можливість спіймати віртуальний об'єкт шляхом натискання на нього на екрані пристрою
- Можливість отримувати нагороди за спіймані віртуальні об'єкти та виконані завдання
- Можливість перегляду досягнень, здобутих у грі, з подальшим використанням у програмах лояльності
- Можливість змінювати налаштування додатку

4.1.3 Для адміністратора системи:

- Можливість додавання нових локацій для розміщення віртуальних об'єктів.
- Можливість додавання нових віртуальних об'єктів у систему та їх прив'язка до локацій.
- Можливість керувати нагородами та досягненнями, що надаються користувачам за взаємодію з віртуальними об'єктами.

4.1.4 Додаткові вимоги:

Не висуваються.

4.2 Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити цілісність інформації в базі даних.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються

4.3.2 Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на мобільних телефонах iPhone.

Мінімальна конфігурація технічних засобів:

- підключення до мережі Інтернет зі швидкістю від 10 мегабіт/с
- 100 Мб вільної пам'яті на пристрої

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем iOS 14.0+.

4.5.1 Вимоги до вхідних даних

Вимоги до вхідних даних не висуваються.

4.5.2 Вимоги до вихідних даних

Вимоги до вихідних даних не висуваються.

4.5.3 Вимоги до мови розробки

Розробку виконати на мові програмування Swift

4.5.4 Вимоги до середовища розробки

Розробку виконати за допомогою середовища Xcode та MySQL Workbench

4.5.5 Вимоги до представленню вихідних кодів

Вимоги до представленню вихідних кодів не висуваються

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування не висуваються

4.8 Спеціальні вимоги

Спеціальні вимоги не висуваються

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- технічне завдання;
- пояснювальна записка;
- текст програми;
- програма та методика тестування;
- керівництво користувача.

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структура варіантів використання;
- схема структурна класів програмного забезпечення;
- схема бази даних.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення літератури за тематикою роботи	21.02	
2.	Розробка технічного завдання	03.03	Технічне завдання
3.	Аналіз вимог та уточнення специфікацій	19.04	Специфікації програмного забезпечення
4.	Проектування структури програмного забезпечення, проектування компонентів	30.04	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів)
5.	Програмна реалізація програмного забезпечення	05.05	Тексти програмного забезпечення
6.	Тестування програмного забезпечення	10.05	Тести, результати тестування
7.	Розробка матеріалів текстової частини роботи	16.05	Пояснювальна записка
8.	Розробка матеріалів графічної частини роботи	20.05	Графічний матеріал проекту
9.	Оформлення технічної документації роботи	29.05	Технічна документація

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

**Пояснювальна записка
до дипломного проєкту**

на тему: Геолокаційна гра з використанням розширеної реальності

КПІ.ІП-0104.045490.02.81

Київ – 2024

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ.....	3
ВСТУП.....	4
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Аналіз предметної області	6
1.2 Аналіз існуючих рішень	7
1.2.1 Аналіз відомих програмних продуктів.....	8
1.2.2 Аналіз відомих алгоритмічних та технічних рішень	10
1.3 Опис бізнес процесів	15
1.4 Постановка задачі	17
Висновки до розділу	18
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	20
2.1 Варіанти використання програмного забезпечення	20
2.2 Аналіз системних вимог	25
2.3 Розроблення функціональних вимог.....	25
2.4 Розроблення нефункціональних вимог.....	28
Висновки до розділу	29
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	31
3.1 Архітектура програмного забезпечення.....	31
3.2 Обґрунтування вибору засобів розробки	36
3.3 Конструювання програмного забезпечення.....	40
3.4 Аналіз безпеки даних.....	49
Висновки до розділу	50
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ..	52
4.1 Аналіз якості ПЗ.....	52

4.2	Опис процесів тестування	54
4.3	Опис контрольного прикладу	61
	Висновки до розділу	67
	5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	69
5.1	Розгортання програмного забезпечення	69
5.2	Супровід програмного забезпечення	70
	Висновки до розділу	71
	ВИСНОВКИ.....	72
	ПЕРЕЛІК ПОСИЛАНЬ	74
	ДОДАТОК А - ЗВІТ ПОДІБНОСТІ.....	77

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

AR - Augmented Reality (Розширена реальність)

GPS - Global Positioning System (Глобальна система позиціонування)

HTTPS - HyperText Transfer Protocol Secure (Захищений протокол передачі гіпертексту)

API - Application Programming Interface (Інтерфейс прикладного програмування)

MVVM - Model-View-ViewModel (Модель-Представлення-Модель представлення, архітектура ПЗ)

PK - Primary Key (Первинний ключ)

FK - Foreign Key (Зовнішній ключ)

IDE - Integrated Development Environment (Інтегроване середовище розробки)

SQL - Structured Query Language (Мова структурованих запитів)

ВСТУП

Мобільні додатки стали невід'ємною частиною повсякденного життя, пропонуючи користувачам широкий спектр функціональних можливостей. Зокрема, геолокаційні додатки та додатки з використанням розширеної реальності (AR) набувають все більшої популярності завдяки своїй здатності інтегрувати віртуальний контент у реальний світ, створюючи нові форми взаємодії та розваг. Розробка геолокаційних ігор з використанням AR дозволяє значно підвищити залученість користувачів та відкриває нові перспективи для розвитку індустрії мобільних ігор[3].

На глобальному рівні, такі компанії як Niantic, Google та Apple активно інвестують у розвиток технологій AR та геолокаційних сервісів. Відомі ігри, такі як Pokémon GO, продемонстрували величезний потенціал інтеграції реального світу з віртуальним через мобільні пристрої[2]. Це стимулює подальший розвиток подібних проектів та розширює можливості для їхнього застосування у різних сферах, від розваг до освіти і туризму.

На сьогоднішній день, ринок AR додатків стрімко розвивається. Провідні наукові установи та технологічні компанії постійно працюють над удосконаленням технологій, що дозволяють покращити якість та реалістичність віртуального контенту. Зокрема, ARKit від Apple надає розробникам потужні інструменти для створення AR додатків, спрощуючи процес інтеграції AR функцій у мобільні додатки на iOS. Провідні вчені та фахівці у галузі комп'ютерних наук та інформаційних технологій активно досліджують нові методи та підходи для покращення взаємодії користувачів з AR контентом.

Геолокаційні ігри з використанням AR можуть знайти застосування у багатьох галузях. Крім індустрії розваг, вони можуть бути корисними в освіті, допомагаючи створювати інтерактивні навчальні матеріали, та у туризмі, пропонуючи нові способи дослідження туристичних місць. Також, такі додатки можуть використовуватись у маркетингу для створення унікальних рекламних кампаній та підвищення залученості клієнтів[3]. У цій роботі буде розглянуто процес розробки простої геолокаційної гри з використанням розширеної

реальності на платформі iOS, що використовує мову програмування Swift та інструментарій SwiftUI. Метою роботи є створення функціонального прототипу, який продемонструє основні можливості та переваги інтеграції геолокаційних та AR технологій у мобільні додатки.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Аналіз предметної області

Предметною областю є розробка геолокаційних ігор з використанням технологій розширеної реальності (AR) на платформі iOS. Геолокаційні ігри є підвидом мобільних ігор, в яких гравець використовує своє фізичне місцезнаходження для взаємодії з ігровим контентом. AR-технології дозволяють інтегрувати віртуальні об'єкти у реальний світ через екран мобільного пристрою, створюючи унікальний ігровий досвід.

Геолокаційні ігри стали популярними завдяки можливостям сучасних мобільних пристроїв, які оснащені GPS-модулями та камерами високої роздільної здатності. Одним з перших успішних прикладів таких ігор є Pokémon GO від компанії Niantic, яка поєднала елементи доповненої реальності з геолокаційними можливостями смартфонів, що дозволило залучити мільйони користувачів по всьому світу[3]. AR-технології, зокрема ARKit від Apple, значно спрощують процес розробки додатків з розширеною реальністю, надаючи розробникам інструменти для точного відслідковування руху, розпізнавання об'єктів та інтеграції віртуальних об'єктів у реальний світ. Це відкрило нові можливості для розвитку ігрової індустрії та інших сфер, таких як освіта, медицина, маркетинг та туризм.

На сьогоднішній день розробка геолокаційних ігор з AR-технологіями базується на інтеграції декількох ключових компонентів: GPS для визначення місцезнаходження, камери для відображення реального світу, AR-рушія для обробки та відображення віртуальних об'єктів, а також програмних інтерфейсів (API) для забезпечення взаємодії між цими компонентами[3]. Процес імплементації геолокаційних функцій зазвичай включає отримання дозволу від користувача на використання його місцезнаходження, обробку даних GPS та визначення координат. AR-компонент забезпечує накладання віртуальних об'єктів на реальний світ, використовуючи дані з камери та сенсорів пристрою[2].

Незважаючи на значний прогрес у розвитку геолокаційних ігор та AR-технологій, існують певні недоліки та обмеження. До основних проблем можна віднести високе споживання енергії мобільних пристроїв під час роботи з GPS та камерою, недостатню точність GPS в умовах міської забудови або приміщень, а також обмеженість обчислювальних ресурсів мобільних пристроїв для обробки складних AR-сцен[10].

Для покращення ситуації у сфері розробки геолокаційних ігор з AR-технологіями можна запропонувати наступні шляхи[15]:

- Використання більш енергоефективних алгоритмів для обробки GPS-даних та роботи з камерою.
- Інтеграція додаткових сенсорів, таких як інерціальні вимірювальні одиниці (IMU), для підвищення точності визначення положення.
- Використання хмарних обчислень для зменшення навантаження на мобільні пристрої.
- Розробка нових методів відображення AR-контенту, що враховують особливості реального середовища.

У рамках моєї роботи буде розроблено просту геолокаційну гру з використанням технологій розширеної реальності на платформі iOS, яка використовує мову програмування Swift та інструментарій SwiftUI. Основна увага буде зосереджена на реалізації базових функцій, таких як визначення місцезнаходження користувача, відображення віртуальних об'єктів у реальному світі та забезпечення взаємодії з цими об'єктами. Вибраний шлях дозволить продемонструвати основні можливості та переваги інтеграції геолокаційних та AR-технологій у мобільні додатки, враховуючи сучасні тенденції та потреби користувачів.

1.2 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації геолокаційної AR-гри. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.2.1 Аналіз відомих програмних продуктів

На ринку існує декілька відомих програмних продуктів, які частково або повністю реалізують функціонал, подібний до того, який планується реалізувати у рамках даного дипломного проєкту. До таких програмних продуктів належать Pokémon GO, Ingress Prime та Geocaching. Кожен з цих додатків має свої особливості та реалізує різний рівень інтеграції геолокаційних функцій та технологій розширеної реальності[2].

Pokémon GO, розроблена компанією Niantic, є однією з найвідоміших геолокаційних ігор з AR-елементами. У цій грі користувачі можуть ловити віртуальних покемонів, які з'являються у реальному світі через екран смартфона. Гра використовує GPS для визначення місцезнаходження гравця та AR для відображення покемонів у реальному середовищі. Гравці також можуть взаємодіяти з покестопами та гімами, які розташовані у визначених місцях на карті[2].

Ingress Prime, також розроблена компанією Niantic, є попередницею Pokémon GO. У цій грі користувачі приєднуються до однієї з двох фракцій і борються за контроль над реальними місцями, що відображаються на карті. Гра використовує геолокацію для визначення місцезнаходження гравця та інтерактивні елементи для взаємодії з ігровим середовищем. Хоча гра не має розширеної реальності в традиційному сенсі, вона створює глибоке інтерактивне середовище через використання геолокації[24].

Geocaching є популярною грою, яка базується на геолокації, але не використовує технології AR. У цій грі користувачі шукають реальні сховища (геокеші), які інші гравці приховують у різних місцях. Геокеш у контексті гри Geocaching — це контейнер, захований в певному місці, координати якого розміщені на спеціалізованому веб-сайті або в мобільному додатку. Геокеш може містити логбук, де гравці залишають записи про знайдення, та інші невеликі предмети для обміну. Процес пошуку геокешу передбачає використання GPS-технології для визначення координат схованки та навігації до неї. Геокешинг поєднує елементи орієнтування, квесту та соціальної взаємодії,

часто включаючи криптографічні підказки та програмні алгоритми для шифрування та дешифрування місць схованок. Використовуючи GPS-координати, гравці можуть знаходити ці сховища та залишати у них свої знахідки або записувати свої відвідування. Geocaching значною мірою залежить від спільноти гравців, які підтримують та оновлюють сховища[12].

Для порівняння дипломного проекту з іншими аналогами можна скористатися таблицею 1.1.

Таблиця 1.1 - Порівняння з аналогами

Функціонал	Pokémon GO	Ingress Prime	Geocaching	Геолокаційна гра з AR (Дипломний проєкт)
Геолокація	Так	Так	Так	Так
Розширена реальність (AR)	Так	Ні	Ні	Так
Інтерактивні об'єкти	Віртуальні об'єкти	Портали	Геокеші	Віртуальні об'єкти
Соціальна взаємодія	Так	Так	Так	Так
Використання GPS	Так	Так	Так	Так
Нагороди та досягнення	Так	Так	Так	Так
Використання в програмах лояльності	Ні	Ні	Ні	Так
Спільнота користувачів	Велика	Середня	Велика	На стадії розробки
Платформа	iOS, Android	iOS, Android	iOS, Android	iOS

1.2.2 Аналіз відомих алгоритмічних та технічних рішень

У процесі створення геолокаційної гри з використанням розширеної реальності для iOS, деякі важливі завдання вимагають розробки ефективних алгоритмічних рішень. Основні з них - обробка геолокаційних даних, інтеграція AR-контенту в реальне середовище і забезпечення користувача можливістю взаємодії з віртуальними об'єктами.

Щодо обробки геолокаційних даних, то серед найбільш поширених алгоритмів, які використовуються для цих цілей, є алгоритм Калмана, SLAM (Simultaneous Localization and Mapping), DBSCAN (Density-Based Spatial Clustering of Applications with Noise) та ORB (Oriented FAST and Rotated BRIEF)[17]. Кожен з цих алгоритмів має свої унікальні особливості та переваги для специфічних завдань AR-додатку. Алгоритм Калмана є інструментом для оцінки стану динамічних систем у реальному часі. Він забезпечує ефективне фільтрування шумів та об'єднання даних з різних сенсорів, що дозволяє досягти високої точності оцінки положення та орієнтації пристрою[5].

SLAM, або одночасна локалізація та побудова карт, є більш комплексним підходом, який дозволяє пристрою одночасно орієнтуватися у просторі та створювати карту цього простору. SLAM використовує дані з сенсорів для побудови карти навколишнього середовища і одночасно визначає положення пристрою відносно цієї карти. Це особливо корисно для робототехніки та автономних транспортних засобів, де пристрій повинен навігувати у невідомому середовищі. Однак, SLAM може бути занадто важким для мобільних пристроїв з обмеженими ресурсами і може вимагати значних обчислювальних потужностей[17].

DBSCAN є алгоритмом кластеризації, який використовується для виявлення груп об'єктів у великій кількості даних. Він добре працює з даними, що містять шум, і може ефективно знаходити кластери довільної форми. У контексті AR-додатку, DBSCAN може бути корисним для аналізу розподілу точок даних, отриманих з сенсорів або зображень, але він не надає прямого рішення для задачі локалізації або відстеження позиції пристрою[17].

ORB (Oriented FAST and Rotated BRIEF) є алгоритмом для виявлення та опису ключових точок у зображеннях. Він використовується для зіставлення зображень та відстеження об'єктів, що є корисним для побудови візуальної одометри та трекінгу у контексті AR-додатків. ORB є ефективним і швидким алгоритмом, але він не забезпечує інтеграції даних з різних сенсорів та не вирішує задачу фільтрування шумів[17].

Порівнявши, стає очевидним, що алгоритм Калмана забезпечує оптимальне рішення для завдань, пов'язаних з фільтруванням шумів та об'єднанням даних з різних сенсорів у реальному часі. Він дозволяє досягти високої точності та стабільності позиціонування, що є критичним для AR-додатків. SLAM, хоча і є потужним інструментом, може бути надмірно важким для мобільних пристроїв. DBSCAN та ORB, хоч і корисні для інших задач, не забезпечують необхідної точності та стабільності для позиціонування та фільтрування даних у AR-додатках.

Розробка геолокаційної гри з AR включає також вибір відповідних технічних рішень, таких як архітектури програмного забезпечення, архітектурні паттерни та платформи. Одним із ключових аспектів є вибір архітектури програмного забезпечення. Архітектури MVC (Model-View-Controller) та MVVM (Model-View-ViewModel) є двома з найбільш використовуваних архітектурних паттернів для побудови мобільних додатків, зокрема у контексті AR-додатків[8]. Обидві архітектури мають свої переваги та недоліки, і їхнє порівняння допоможе визначити найбільш підходящий підхід для розробки конкретного AR-додатку.

Архітектура MVC розділяє додаток на три основні компоненти: модель, представлення та контролер. Модель відповідає за управління даними та бізнес-логікою, представлення забезпечує інтерфейс користувача, а контролер виступає посередником між моделлю та представленням, обробляючи ввід користувача та оновлюючи відповідні компоненти. У контексті AR-додатку, модель може включати дані про користувача, ігрові об'єкти та їхні властивості. Представлення складається з інтерфейсу, що відображає AR-об'єкти та взаємодії з ними[7].

Контролер відповідає за обробку подій, таких як дії користувача або зміни у стані сенсорів. Основною перевагою MVC є чітке розділення відповідальностей, що спрощує розробку, тестування та підтримку коду. Однак, з часом контролери можуть стати перевантаженими логікою, що призводить до труднощів у підтримці та розширенні додатку[8]. У контексті AR-додатку це може бути особливо проблематично, оскільки обробка взаємодії з AR-об'єктами та обробка сенсорних даних можуть створювати значний обсяг коду в контролері.

Архітектура MVVM пропонує альтернативний підхід, розділяючи додаток на модель, представлення та модель представлення. Модель представлення виступає посередником між моделлю та представленням, забезпечуючи двостороннє зв'язування даних та відокремлюючи логіку від інтерфейсу користувача. У контексті AR-додатку, модель представлення обробляє логіку, пов'язану з управлінням станом AR-об'єктів та обробкою взаємодії з користувачем, тоді як представлення відповідає лише за відображення інтерфейсу. Основною перевагою MVVM є можливість легкого тестування та масштабованості коду завдяки чіткому розділенню логіки та інтерфейсу користувача. Двостороннє зв'язування даних спрощує процес синхронізації стану моделі з інтерфейсом, що особливо корисно у динамічних AR-додатках, де зміни в реальному світі можуть швидко впливати на віртуальний контент. Крім того, MVVM дозволяє зменшити обсяг коду у представленнях, роблячи їх більш легкими та підтримуваними. У порівнянні з MVC, архітектура MVVM надає більше можливостей для рефакторингу та розширення додатку без значних змін у кодовій базі. Це робить MVVM більш підходящою для великих проектів та командної роботи. Однак, MVVM може вимагати більшої кількості початкового налаштування та використання додаткових інструментів для зв'язування даних, що може збільшити складність на ранніх етапах розробки[8].

У додатку також можуть бути використані різні архітектурні паттерни, такі як Observer та Singleton. Паттерн Observer використовується для встановлення механізму підписки, де один об'єкт (спостерігач) отримує сповіщення про зміни стану іншого об'єкта (суб'єкта). У контексті AR-додатку цей паттерн може бути

корисним для обробки подій, що походять від сенсорів пристрою, таких як зміни в положенні або орієнтації. Наприклад, AR-додаток може мати об'єкт, який відповідає за збір даних з сенсорів, і декілька об'єктів-спостерігачів, які оновлюються у відповідь на зміни цих даних[7]. Це дозволяє забезпечити динамічне оновлення віртуальних об'єктів відповідно до реальних змін у середовищі користувача. Основною перевагою паттерну Observer є його здатність забезпечувати слабке зв'язування між компонентами, що підвищує гнучкість та масштабованість системи. Однак, зростання кількості спостерігачів може призвести до збільшення складності управління підписками та сповіщеннями, що потребує додаткових механізмів для ефективного керування ресурсами[8].

Паттерн Singleton, з іншого боку, гарантує, що клас має лише один екземпляр і надає глобальну точку доступу до цього екземпляра. У контексті AR-додатку Singleton може бути використаний для управління ресурсами, що мають бути доступні унікально у всьому додатку, такими як налаштування AR-сесії або доступ до камери. Це забезпечує централізований контроль та запобігає створенню кількох екземплярів об'єкта, що може призвести до конфліктів або надмірного використання ресурсів. Основна перевага паттерну Singleton полягає у його простоті та ефективності в забезпеченні унікальності ресурсу. Однак, надмірне використання Singleton може призвести до сильної залежності між компонентами та ускладнити тестування, оскільки глобальний стан системи стає важче ізолювати[8]. У контексті AR-додатку обидва паттерни можуть бути використані для різних цілей. Observer підходить для ситуацій, де необхідно забезпечити реактивність системи на зміну стану середовища або користувацькі дії, завдяки можливості динамічного оновлення компонентів у відповідь на події[7]. Це дозволяє створювати більш інтерактивні та адаптивні AR-додатки. Singleton, в свою чергу, корисний для управління унікальними ресурсами та забезпечення їхньої доступності у всьому додатку, що є критичним для ефективного використання ресурсів пристрою.

У розробці AR-ігор також важливо обирати правильні платформи та інструменти. ARKit від Apple може бути використаний для створення додатків з AR, забезпечуючи високий рівень точності трекінгу та простоту інтеграції. З іншого боку, для роботи з геолокаційними даними можна використовувати фреймворк CoreLocation, який забезпечує точність та надійність. З технічної точки зору, ARKit надає набір API для обробки камер, сенсорів руху та інших даних пристрою для створення AR-досвіду, тоді як CoreLocation забезпечує API для доступу до даних GPS та інших геолокаційних сенсорів.

Наведемо таблицю 1.2. для більш наочного порівняння згаданих технічних рішень.

Таблиця 1.2 - Порівняльний аналіз технічних рішень

Компонент	Рішення	Переваги	Недоліки
Архітектура ПЗ	MVC	Простота реалізації	Менша гнучкість
	MVVM	Краща підтримка та тестованість	Складніша реалізація
Архітектурні паттерни	Observer	Гнучкість	Можлива складність управління
	Singleton	Ефективність управління ресурсами	Можливі проблеми з тестуванням
Компонент	Рішення	Переваги	Недоліки
Платформи	ARKit	Висока точність, інтеграція з iOS	Обмеження платформи iOS
	CoreLocation	Надійність, підтримка від Apple	Обмеження платформи iOS

На основі проведеного аналізу, у рамках дипломного проекту буде використана архітектура MVVM, оскільки вона забезпечує кращу підтримку та тестованість додатка. Для інтеграції AR та геолокаційних функцій будуть

використані ARKit та CoreLocation відповідно, оскільки ці інструменти забезпечують високу точність та надійність.

1.3 Опис бізнес процесів

Для опису бізнес процесів скористаємося BPMN моделлю (рисунк 1.2-1.4)

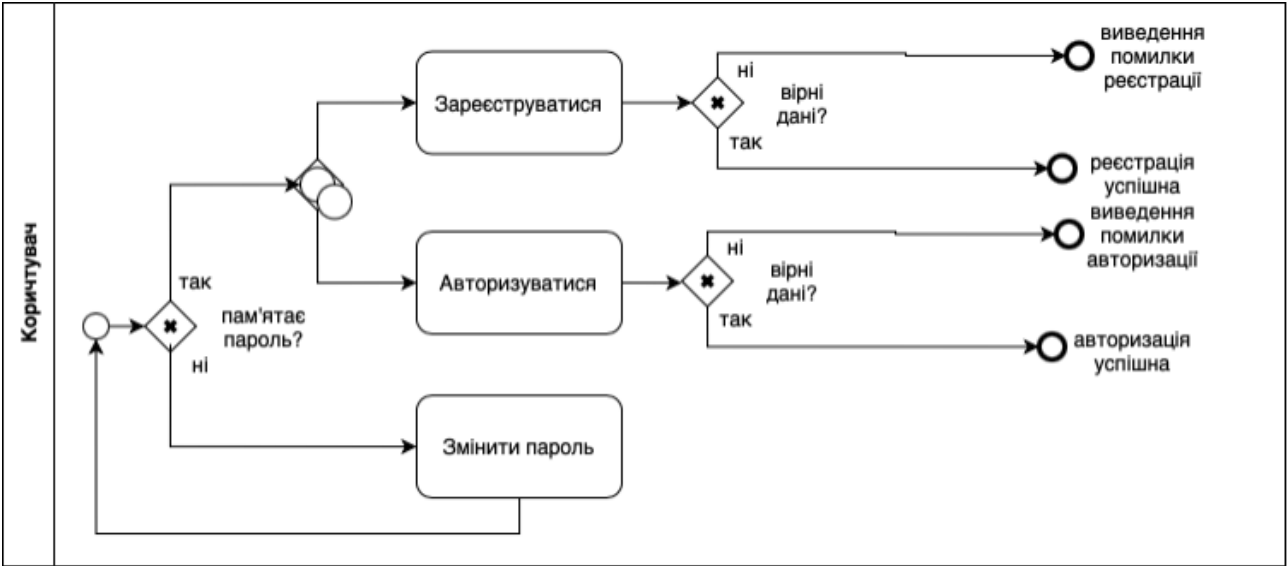


Рисунок 1.1 - Схема бізнес процесу дій з реєстраційними даними

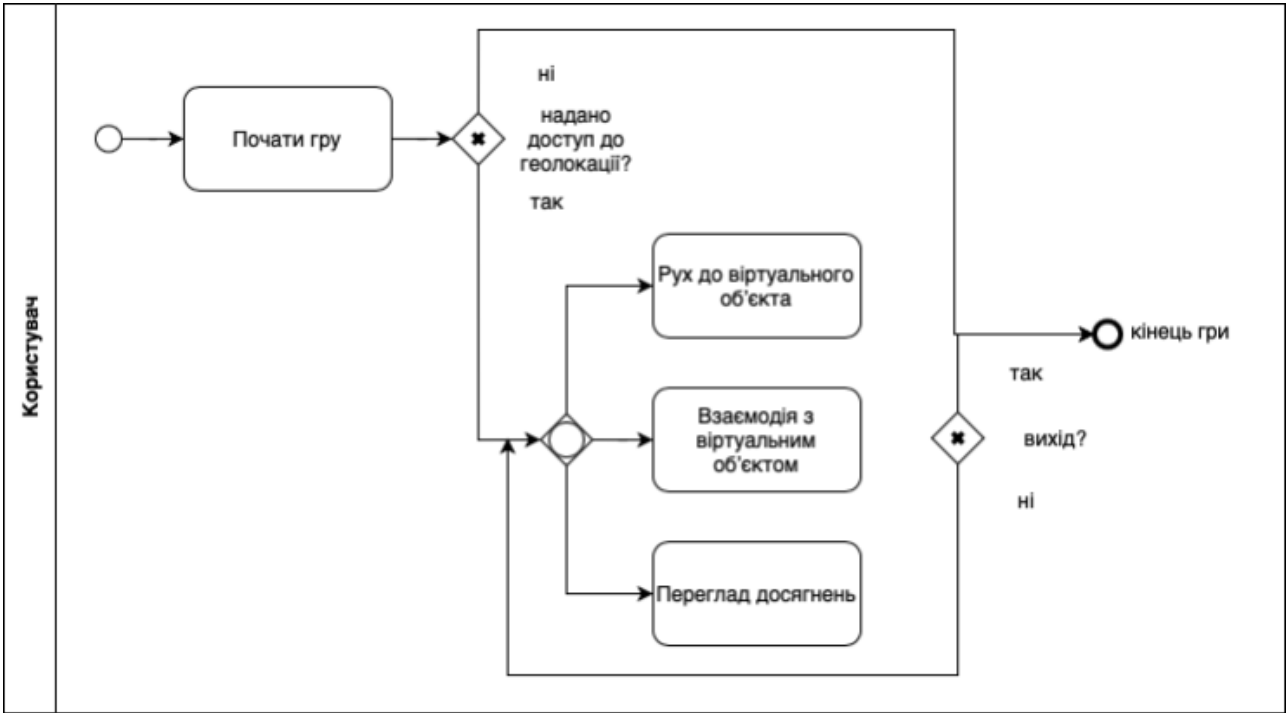


Рисунок 1.2 - Схема бізнес процесу геолокаційної гри з доповненою реальністю

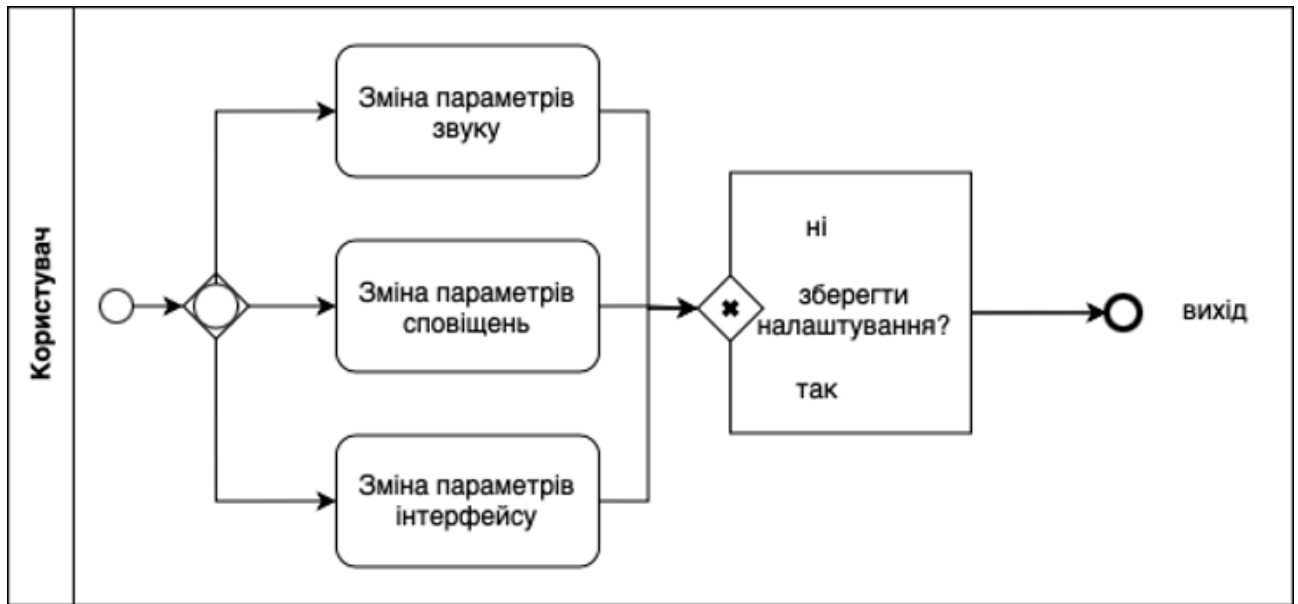


Рисунок 1.3 - Схема бізнес процесу зміни налаштувань

Опис послідовності реєстрації та авторизації:

- Користувач відкриває додаток
- Користувач обирає зареєструватися або авторизуватися
- Якщо користувач реєструється, то відбувається введення даних (ім'я, email, пароль) та підтвердження реєстрації. Якщо дані вірні, користувача реєструють. Інакше виводиться помилка
- Якщо користувач авторизується, то відбувається введення логіна та пароля та підтвердження авторизації. Якщо дані вірні, користувач потрапляє на головний екран. Інакше виводиться помилка

Опис послідовності зміни паролю:

- Користувач натискає на кнопку "Змінити пароль".
- Користувач переходить на екран зміни паролю.
- Користувач вводить свою поштову адресу. Якщо адреса введена правильно, користувач переходить на наступний екран, і на його поштову скриньку надсилається код. У випадку неправильно введеної адреси на екрані з'являється повідомлення про помилку.
- Користувач вводить код. Якщо код введено правильно, користувача переадресовує на наступний екран. У протилежному випадку виводиться помилка.

– Користувач встановлює новий пароль та повторно його вводить для підтвердження. Якщо обидва введені паролі співпадають, то зміна паролю вважається успішною. У протилежному випадку виводиться повідомлення про помилку.

Опис послідовності запуску гри:

- Користувач обирає почати гру
- Додаток запитує дозвіл на доступ до геолокації
- Додаток визначає місцезнаходження користувача
- На карті відображаються цілі

Опис послідовності взаємодії з віртуальними об'єктами:

- Користувач рухається до віртуального об'єкта
- Додаток використовує ARKit для відображення об'єкта
- Користувач бачить об'єкт у реальному світі через камеру
- Користувач взаємодіє з об'єктом (наприклад, збирає його)

Опис послідовності отримання нагород та досягнень:

- Після взаємодії з об'єктом, користувач отримує нагороду
- Додаток оновлює досягнення користувача
- Користувач може переглянути свої досягнення

Опис послідовності взаємодії з налаштуваннями:

- Користувач обирає меню налаштувань
- Користувач змінює параметри гри (звуки, сповіщення, інтерфейс)
- Додаток зберігає налаштування

1.4 Постановка задачі

Метою моєї дипломної роботи є створення геолокаційної гри з використанням технологій розширеної реальності (AR) на платформі iOS, яка дозволяє користувачам інтерактивно взаємодіяти з віртуальними об'єктами у реальному світі. Розробка такого додатку спрямована на демонстрацію можливостей сучасних мобільних технологій, зокрема, інтеграції геолокаційних

сервісів та AR для створення захоплюючого ігрового досвіду. Основні цілі розробки включають:

- Створення стабільного та надійного мобільного додатку для платформи iOS, який використовує можливості ARKit та CoreLocation для відображення віртуальних об'єктів у реальному світі.
- Забезпечення точного визначення місцезнаходження користувача для інтерактивної взаємодії з ігровим контентом.
- Розробка механізмів для взаємодії користувача з віртуальними об'єктами, включаючи збір нагород та досягнень.
- Реалізація інтуїтивно зрозумілого інтерфейсу користувача, який забезпечує простоту використання та залученість.

На виході очікується готовий мобільний додаток для платформи iOS, що забезпечує інтерактивну взаємодію користувача з віртуальними об'єктами у реальному світі.

Висновки до розділу

У цьому розділі було проведено всебічний аналіз предметної області, зокрема, розглянуто основні риси геолокаційних ігор з використанням технологій розширеної реальності (AR) на платформі iOS. Було описано загальні положення та напрямки розвитку цієї галузі, а також проведено детальний огляд сучасних алгоритмічних та технічних рішень, які використовуються для розробки подібних додатків.

Перш за все, було визначено актуальність роботи, обґрунтовано необхідність розробки геолокаційної гри з AR та розглянуто світові тенденції у цій галузі. Було описано сучасний стан об'єкта розробки, проаналізовано існуючі рішення від провідних наукових установ та компаній, а також визначено можливі сфери застосування розробки. Відповідно до поставлених задач, було проведено аналіз відомих програмних продуктів, які частково або повністю реалізують подібний функціонал. До таких продуктів належать Pokémon GO, Ingress Prime та Geocaching. Було створено порівняльну таблицю функціоналу та особливостей цих продуктів у порівнянні з розроблюваним додатком. Це

дозволило визначити сильні та слабкі сторони кожного рішення та обґрунтувати вибір технологій та підходів для нашого проєкту.

Також було розглянуто відомі алгоритмічні та технічні рішення для вирішення задач обробки геолокаційних даних, інтеграції AR-контенту та взаємодії з користувачем. Було проведено порівняльний аналіз алгоритмів, таких як алгоритм Калмана, DBSCAN, SLAM та ORB, а також архітектурних патернів MVC та MVVM. Це дозволило визначити найбільш підходящі алгоритми та технічні рішення для реалізації поставлених задач. На основі проведеного аналізу було сформульовано мету, цілі та задачі розробки, визначено вхідні дані та очікувані результати.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є взаємодія з віртуальними об'єктами в реальному світі через камеру. Більше функцій наведено в графічному матеріалі роботи.

В таблицях 2.1 – 2.11 наведені варіанти використання програмного забезпечення.

Таблиця 2.1 - Варіант використання UC-1

Use case name	Реєстрація
Use case ID	UC-01
Goals	Реєстрація нового користувача у системі
Actors	Користувач
Trigger	Користувач бажає зареєструватися
Pre-conditions	-
Flow of Events	Користувач відкриває застосунок, обирає реєстрацію, вводить ім'я, email та пароль, натискає кнопку «Зареєструватися»
Extension	У разі помилки введених даних або з'єднання з сервером має з'явитися вікно помилки.
Post-Condition	Користувача реєструють у системі та переводять на головний екран

Таблиця 2.2 - Варіант використання UC-2

Use case name	Авторизація
Use case ID	UC-02
Goals	Авторизація користувача у системі
Actors	Користувач
Trigger	Користувач бажає авторизуватися
Pre-conditions	-

Продовження таблиці 2.2

Flow of Events	Користувач відкриває застосунок, вводить логін та пароль, натискає кнопку «Увійти»
Extension	У разі помилки з'єднання з сервером або невірно введених даних логіну та паролю має з'явитися вікно помилки.
Post-Condition	Користувача переводить на основний екран

Таблиця 2.3 - Варіант використання UC-3

Use case name	Зміна паролю
Use case ID	UC-03
Goals	Зміна паролю користувача
Actors	Користувач
Trigger	Користувач бажає змінити пароль
Pre-conditions	Користувач має бути авторизованим
Flow of Events	Користувач натискає кнопку "Змінити пароль", вводить поштову адресу, вводить отриманий код, вводить новий пароль та підтверджує його
Extension	У разі помилки введених даних або невірного коду має з'явитися вікно помилки.
Post-Condition	Пароль користувача успішно змінено

Таблиця 2.4 - Варіант використання UC-4

Use case name	Почати гру
Use case ID	UC-04
Goals	Початок гри
Actors	Користувач
Trigger	Користувач бажає почати гру
Pre-conditions	Користувач має бути авторизованим
Flow of Events	Користувач обирає почати гру, додаток запитує дозвіл на доступ до геолокації, визначає місцезнаходження користувача, на карті відображаються цілі

Продовження таблиці 2.4

Extension	У разі відмови у доступі до геолокації або помилки визначення місцезнаходження має з'явитися вікно помилки
Post-Condition	Гра розпочинається, користувач бачить цілі на карті

Таблиця 2.5 - Варіант використання UC-5

Use case name	Взаємодія з віртуальними об'єктами
Use case ID	UC-05
Goals	Взаємодія користувача з віртуальними об'єктами у грі
Actors	Користувач
Trigger	Користувач знаходить віртуальний об'єкт
Pre-conditions	Користувач має бути авторизованим та почати гру
Flow of Events	Користувач рухається до віртуального об'єкта, додаток використовує ARKit для відображення об'єкта, користувач взаємодіє з об'єктом
Extension	У разі помилки у роботі ARKit має з'явитися вікно помилки
Post-Condition	Користувач успішно взаємодіє з об'єктом

Таблиця 2.6 - Варіант використання UC-6

Use case name	Спіймати віртуальний об'єкт при натисканні
Use case ID	UC-06
Goals	Спіймати віртуальний об'єкт у грі
Actors	Користувач
Trigger	Користувач натискає на віртуальний об'єкт
Pre-conditions	Користувач авторизований, знаходиться у грі та взаємодіє з віртуальними об'єктами
Flow of Events	Користувач натискає на віртуальний об'єкт, система реєструє натискання і взаємодію, об'єкт спіймано.
Extension	У разі помилки, система виводить повідомлення про помилку
Post-Condition	Користувач успішно спіймав віртуальний об'єкт

Таблиця 2.7 - Варіант використання UC-7

Use case name	Отримати нагороду
Use case ID	UC-07
Goals	Отримання нагороди за виконання завдання
Actors	Користувач
Trigger	Користувач завершує завдання
Pre-conditions	Користувач авторизований та знаходиться у грі
Flow of Events	Користувач завершує завдання у грі, система нараховує нагороду і повідомляє користувача про це.
Extension	У разі помилки, система виводить повідомлення про помилку
Post-Condition	Користувач отримує нагороду

Таблиця 2.8 - Варіант використання UC-8

Use case name	Перегляд досягнень
Use case ID	UC-08
Goals	Перегляд досягнень та нагород користувача
Actors	Користувач
Trigger	Користувач бажає переглянути свої досягнення
Pre-conditions	Користувач має бути авторизованим
Flow of Events	Користувач обирає перегляд досягнень, додаток відображає список досягнень та нагород
Extension	У разі помилки у завантаженні даних має з'явитися вікно помилки
Post-Condition	Користувач переглядає свої досягнення та нагороди

Таблиця 2.9 - Варіант використання UC-9

Use case name	Налаштування
Use case ID	UC-09
Goals	Зміна налаштувань гри
Actors	Користувач

Продовження таблиці 2.9

Trigger	Користувач бажає змінити налаштування гри
Pre-conditions	Користувач має бути авторизованим
Flow of Events	Користувач обирає меню налаштувань, змінює параметри гри (звуки, сповіщення, інтерфейс)
Extension	У разі помилки у збереженні налаштувань має з'явитися вікно помилки
Post-Condition	Зміни в налаштуваннях успішно збережені

Таблиця 2.10 - Варіант використання UC-10

Use case name	Додавання локацій
Use case ID	UC-10
Goals	Додавання нових локацій до гри
Actors	Адміністратор
Trigger	Адміністратор обирає додавання нової локації
Pre-conditions	Адміністратор авторизований
Flow of Events	Адміністратор вводить дані нової локації та зберігає їх у системі
Extension	У разі помилки, система виводить повідомлення про помилку
Post-Condition	Нова локація успішно додана до гри

Таблиця 2.11 - Варіант використання UC-11

Use case name	Додавання віртуальних об'єктів
Use case ID	UC-11
Goals	Додавання нових віртуальних об'єктів до гри
Actors	Адміністратор
Trigger	Адміністратор обирає додавання нового віртуального об'єкта
Pre-conditions	Адміністратор авторизований
Flow of Events	Адміністратор вводить дані нового віртуального об'єкта та зберігає їх у системі

Продовження таблиці 2.11

Extension	У разі помилки, система виводить повідомлення про помилку
Post-Condition	Новий віртуальний об'єкт успішно доданий до гри

2.2 Аналіз системних вимог

Для забезпечення функціонування мобільного додатку для геолокаційної гри з використанням розширеної реальності, пристрій користувача повинен мати операційну систему iOS 14.0 або новішу версію. Мобільний додаток вимагає наявності стабільного інтернет-з'єднання зі швидкістю не менше 10 мегабіт/сек, щоб забезпечити своєчасне завантаження та оновлення даних, пов'язаних з ігровими об'єктами та геолокаційними даними. Для коректного відображення віртуальних об'єктів та інтерактивності гри, пристрій повинен мати вбудований GPS-модуль, а також підтримку технології ARKit, що є стандартом для пристроїв Apple, починаючи з iPhone 6S і новіше. Окрім того, пристрій має мати не менше 100 МБ вільної пам'яті для забезпечення швидкого оброблення даних та стабільної роботи додатку без збоїв. Серверна частина додатку базується на фреймворку Swift Vapor, який забезпечує високопродуктивне середовище для створення веб-додатків і API на мові Swift[23]. Сервер повинен бути розгорнутий на операційній системі, яка підтримує виконання Swift, такої як macOS або Ubuntu з налаштованим середовищем для роботи з Swift.

2.3 Розроблення функціональних вимог

Програмне забезпечення розділене на модулі, кожен з яких має свій функціонал. Таблиця 2.12 містить загальну модель вимог, а таблиця 2.13 має деталізований опис цих функціональних вимог до програмного забезпечення. На рисунку 2.1 зображена матриця трасування вимог.

Таблиця 2.12 - Загальна модель вимог

№	Назва	ID вимоги	Пріоритети	Ризики
1	Реєстрація	FR-1	Високий	Високий
2	Авторизація	FR-2	Високий	Високий
3	Зміна паролю	FR-3	Середній	Високий
3.1	Введення пошти та надсилання коду	FR-4	Середній	Високий
3.2	Перевірка коду	FR-5	Середній	Високий
4	Початок гри	FR-6	Високий	Високий
5	Визначення місцезнаходження	FR-7	Високий	Високий
6	Відображення цілей на карті	FR-8	Високий	Високий
7	Взаємодія з віртуальними об'єктами	FR-9	Високий	Високий
7.1	Спіймати віртуальний об'єкт при натисканні	FR-10	Високий	Високий
7.2	Отримання нагород	FR-11	Середній	Середній
8	Перегляд досягнень	FR-12	Середній	Середній
9	Налаштування	FR-13	Середній	Середній
10	Додавання локацій	FR-14	Високий	Високий
11	Додавання віртуальних об'єктів	FR-15	Високий	Високий

Таблиця 2.13 - Перелік функціональних вимог

ID вимоги	Назва та опис
FR-1	Реєстрація Користувач вводить свої дані (ім'я, email, пароль) для реєстрації у системі.

Продовження таблиці 2.13

ID вимоги	Назва та опис
FR-2	Авторизація Користувач вводить логін та пароль для входу у систему.
FR-3	Зміна паролю Користувач може змінити свій пароль через введення старого та нового паролю.
FR-4	Введення пошти та надсилання коду Користувач вводить свою email адресу для отримання коду підтвердження.
FR-5	Перевірка коду Користувач вводить отриманий код для підтвердження.
FR-6	Початок гри Користувач обирає опцію початку гри.
FR-7	Визначення місцезнаходження Додаток визначає поточне місцезнаходження користувача.
FR-8	Відображення цілей на карті Додаток відображає цілі на карті в залежності від місцезнаходження користувача.
FR-9	Взаємодія з віртуальними об'єктами Користувач взаємодіє з віртуальними об'єктами через додаток.
FR-10	Спіймати віртуальний об'єкт при натисканні Користувач може спіймати віртуальний об'єкт при натисканні на нього.
FR-11	Отримання нагород Користувач отримує нагороду за виконання певних дій у грі.
FR-12	Перегляд досягнень Користувач може переглядати свої досягнення у додатку.

Продовження таблиці 2.13

ID вимоги	Назва та опис
FR-13	Налаштування Користувач може змінювати налаштування гри.
FR-14	Додавання локацій Адміністратор може додавати нові локації у додаток.
FR-15	Додавання віртуальних об'єктів Адміністратор може додавати нові віртуальні об'єкти у додаток.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	FR-14	FR-15
UC-1	+	+	+	+	+										
UC-2		+	+	+	+	+	+								
UC-3							+	+							
UC-4								+	+						
UC-5									+	+					
UC-6										+	+				
UC-7											+	+			
UC-8												+	+		
UC-9													+	+	
UC-10														+	+
UC-11															+

Рисунок 2.1 - Матриця трасування вимог

2.4 Розроблення нефункціональних вимог

Мобільний додаток повинен функціонувати на мобільних пристроях з операційною системою iOS 14.0 і старше. Додаток має бути сумісним з моделями iPhone, які підтримують технологію ARKit, що включає iPhone 6s і новіші моделі. Час відгуку додатку не повинен перевищувати 2 секунд при виконанні основних операцій, таких як авторизація, завантаження карти та віртуальних об'єктів. Додаток повинен забезпечувати стабільну роботу при одночасному відображенні до 50 віртуальних об'єктів на карті. Максимальний час визначення місцезнаходження користувача після запуску гри не повинен перевищувати 5

секунд. Додаток повинен бути стійким до мережевих збоїв та забезпечувати коректне відновлення роботи після відновлення інтернет-з'єднання. У разі відсутності доступу до інтернету додаток повинен зберігати основну функціональність, таку як перегляд локально збережених досягнень та налаштувань. Додаток повинен забезпечувати шифрування переданих даних між клієнтом та сервером для захисту персональних даних користувачів. Реєстрація та авторизація користувачів повинні виконуватись із застосуванням безпечних протоколів аутентифікації, таких як OAuth 2.0. Додаток повинен зберігати локально не більше 100 МБ даних користувача, включаючи досягнення та налаштування. Серверна частина повинна забезпечувати резервне копіювання даних користувачів щодня. Інтерфейс додатку повинен бути адаптивним та підтримувати різні розміри екранів мобільних пристроїв. Інтерфейс має бути інтуїтивно зрозумілим та забезпечувати легкий доступ до основних функцій додатку, таких як початок гри, перегляд досягнень та налаштування. Для повноцінного функціонування додатку потрібно інтернет-з'єднання зі швидкістю від 10 мегабіт/с. Додаток повинен мати механізми для перевірки стабільності інтернет-з'єднання та повідомляти користувача про можливі проблеми зі з'єднанням.

Висновки до розділу

У даному розділі було виконано всебічний аналіз та розробку вимог до програмного забезпечення геолокаційної гри з використанням розширеної реальності на платформі iOS. Спочатку було визначено основні функціональні вимоги, які включають процеси реєстрації, авторизації, зміни паролю, початку гри, взаємодії з віртуальними об'єктами, перегляду досягнень та налаштувань. Кожен з цих процесів було детально описано, зокрема розроблено відповідні варіанти використання та сформульовано конкретні вимоги до кожної функції. Всі варіанти використання були представлені у вигляді таблиць, які містять докладний опис кроків виконання, умов початку та завершення, а також можливі розширення у разі виникнення помилок.

Крім того, було розроблено матрицю трасування вимог, яка показує відповідність між функціональними вимогами та варіантами використання. Ця матриця допомагає забезпечити, що всі вимоги враховані та реалізовані у відповідних сценаріях використання, що дозволяє уникнути можливих пропусків та невідповідностей у процесі розробки. Особливу увагу було приділено нефункціональним вимогам, які включають вимоги до платформної сумісності, продуктивності, надійності, безпеки, зберігання даних, інтерфейсу користувача та мережевих з'єднань. Ці вимоги встановлюють стандарти та обмеження, які мають забезпечити стабільну, надійну та безпечну роботу додатку на різних пристроях і в умовах різних мережевих середовищ.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Основними компонентами системи є мобільний додаток, сервер та база даних. Мобільний додаток відповідає за взаємодію з користувачем, обробку даних геолокації та відображення віртуальних об'єктів за допомогою технології ARKit. Сервер забезпечує централізоване зберігання даних користувачів, обробку запитів від клієнтських додатків та управління ігровою логікою. База даних зберігає інформацію про користувачів, їх досягнення, місцезнаходження віртуальних об'єктів та інші необхідні дані. Для забезпечення зв'язку між компонентами системи використовується протокол HTTP/HTTPS[23]. Сервер взаємодіє з мобільним додатком через API, що дозволяє обробляти запити від клієнтів та відправляти відповідні дані. База даних підключена до сервера через протокол TCP/IP, що забезпечує надійне та швидке зберігання і доступ до даних[23]. Нижче наведено діаграму компонентів, яка ілюструє зв'язки між основними компонентами системи(рисунк 3.1).

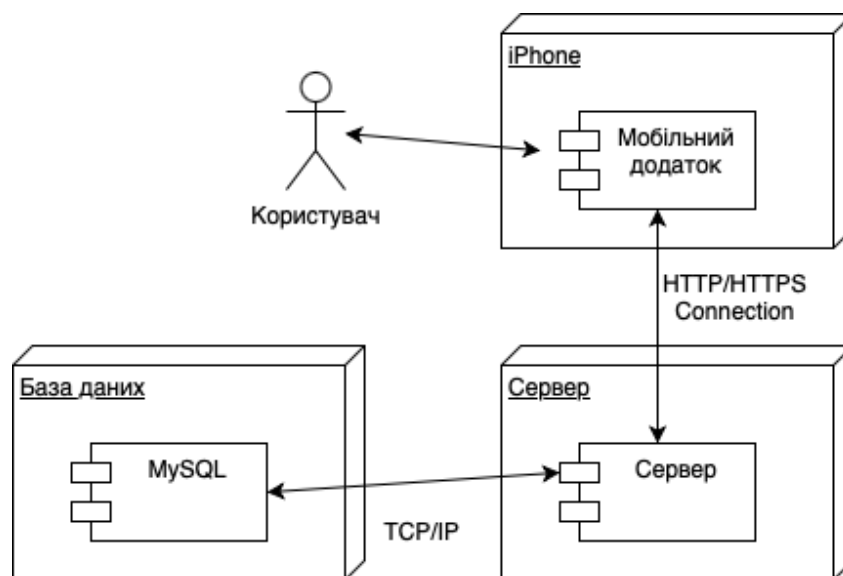


Рисунок 3.1 - Діаграма компонентів

Діаграми послідовностей представлені на рисунках 3.2-3.4



Рисунок 3.2 - Діаграма послідовностей при авторизації

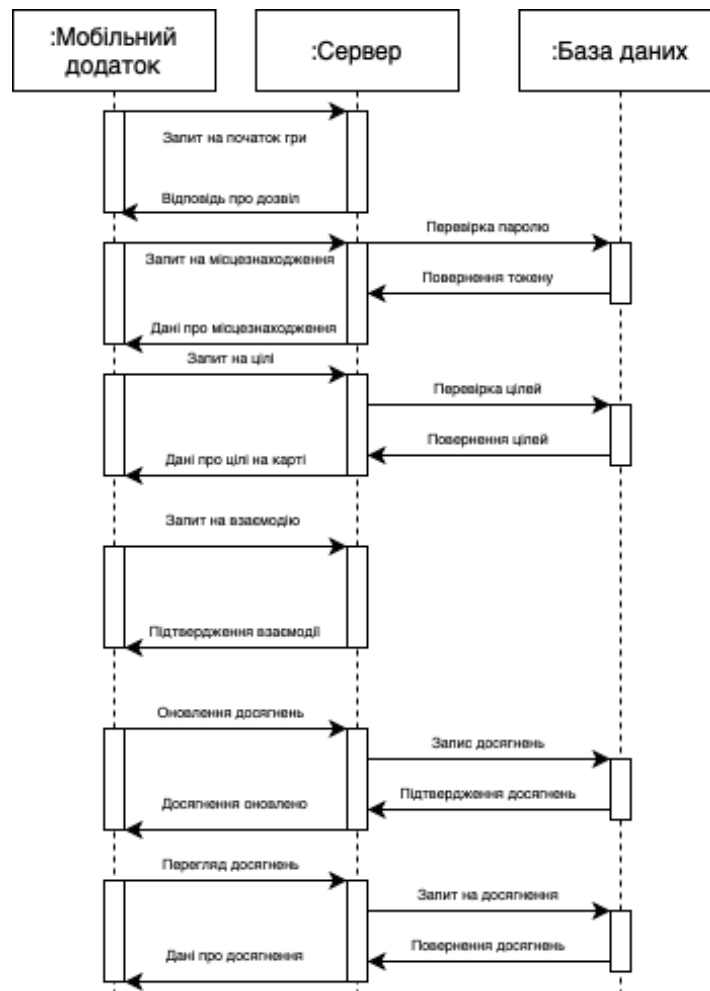


Рисунок 3.3 - Діаграма послідовностей основних функцій



Рисунок 3.4 - Діаграма послідовностей другорядних функцій

Мобільний додаток надсилає запит на валідність сервера, сервер відповідає підтвердженням. Додаток надсилає запит на авторизацію, сервер перевіряє логін та пароль через базу даних. База даних перевіряє логін та пароль і повертає результат перевірки серверу. Якщо логін та пароль вірні, сервер повертає токен авторизації мобільному додатку. Додаток отримує токен і надсилає запит на оновлення токenu. Сервер перевіряє валідність токenu через базу даних і повертає підтвердження мобільному додатку. Для додавання користувача, мобільний додаток надсилає дані нового користувача до сервера. Сервер перевіряє дані і додає нового користувача до бази даних. База даних підтверджує додавання користувача і сервер надсилає підтвердження до мобільного додатку.

Процес зміни паролю починається з того, що мобільний додаток надсилає запит на зміну паролю до сервера. Сервер відправляє код підтвердження на електронну пошту користувача. Користувач вводить отриманий код у мобільний додаток. Сервер перевіряє код та оновлює пароль у базі даних. Сервер надсилає підтвердження успішної зміни паролю до мобільного додатку.

Для перегляду досягнень мобільний додаток надсилає запит на перегляд досягнень до сервера. Сервер отримує дані з досягненнями з бази даних. Сервер надсилає дані з досягненнями до мобільного додатку. Ця діаграма послідовностей відображає взаємодію між мобільним додатком, сервером та

базою даних при виконанні другорядних функцій, забезпечуючи їх коректну реалізацію та роботу системи в цілому.

Процес початку гри включає надсилення мобільним додатком запиту на початок гри до сервера. Сервер відповідає про дозвіл на початок гри. Мобільний додаток надсилає запит на визначення місцезнаходження. Сервер отримує дані про місцезнаходження і надсилає їх мобільному додатку. Для відображення цілей на карті мобільний додаток надсилає запит на отримання цілей до сервера. Сервер надсилає дані про цілі. Мобільний додаток відображає цілі на карті. Взаємодія з віртуальними об'єктами починається з того, що мобільний додаток надсилає запит на взаємодію з віртуальним об'єктом до сервера. Сервер підтверджує взаємодію з об'єктом. Мобільний додаток надсилає запит на оновлення досягнень до сервера. Сервер підтверджує оновлення досягнень.

Геолокаційна гра з використанням розширеної реальності на платформі iOS використовує архітектурний шаблон Model-View-ViewModel (MVVM), що забезпечує розподілення відповідальності між компонентами додатку та спрощує його розширення та підтримку. Модель включає всі дані та логіку, необхідні для функціонування додатку. Вона відповідає за управління даними, отримання їх з бази даних або сервера та забезпечення бізнес-логіки. Основні компоненти моделі включають:

- Дані про користувачів: логіни, паролі, профілі, досягнення.
- Дані про місцезнаходження віртуальних об'єктів.
- Дані про ігрові сесії та результати взаємодії з віртуальними об'єктами.

Представлення відповідає за відображення даних користувачу та забезпечення інтерфейсу для взаємодії з додатком. Воно включає всі UI-компоненти, такі як кнопки, карти, візуальні елементи AR та інші елементи, що забезпечують користувацький досвід. Основні функції представлення включають:

- Відображення поточного місцезнаходження користувача на карті.

- Відображення віртуальних об'єктів у реальному світі за допомогою ARKit.
- Забезпечення інтерфейсу для реєстрації, авторизації, зміни паролю та перегляду досягнень.

Модель-представлення забезпечує зв'язок між моделлю та представленням. Вона відповідає за отримання даних з моделі, обробку цих даних та їх підготовку для відображення у представленні. ViewModel також обробляє дії користувача та надсилає відповідні команди до моделі. Основні завдання ViewModel включають:

- Обробка даних про місцезнаходження користувача та підготовка їх для відображення на карті.
- Отримання даних про віртуальні об'єкти та підготовка їх для відображення за допомогою ARKit.
- Обробка запитів на реєстрацію, авторизацію, зміну паролю та взаємодію з віртуальними об'єктами.
- Забезпечення оновлення представлення відповідно до змін у даних моделі.

У рамках архітектури MVVM, взаємодія між компонентами забезпечується через чітко визначені інтерфейси та зв'язки. Представлення взаємодіє з ViewModel, який у свою чергу взаємодіє з моделлю. Це дозволяє розділити відповідальність та знизити рівень зв'язаності між компонентами, що спрощує тестування та підтримку додатку. Мобільний додаток, сервер та база даних взаємодіють між собою для забезпечення функціональності додатку. Мобільний додаток надсилає запити до сервера через HTTP/HTTPS, сервер обробляє ці запити та взаємодіє з базою даних через TCP/IP для отримання та зберігання необхідних даних[19]. Ця архітектура забезпечує високу надійність, масштабованість та безпеку системи, дозволяючи ефективно обробляти дані та забезпечувати безперебійну роботу додатку. Для забезпечення високої продуктивності додатку реалізовано оптимізацію обробки запитів та взаємодії з сервером. Час відгуку додатку не повинен перевищувати 2 секунд при виконанні

основних операцій, таких як авторизація, завантаження карти та віртуальних об'єктів. Використання кешування даних на стороні клієнта дозволяє зменшити кількість запитів до сервера і підвищити швидкість роботи додатку. Максимальний час визначення місцезнаходження користувача після запуску гри не повинен перевищувати 5 секунд, що досягається завдяки ефективному використанню CoreLocation та ARKit[7].

3.2 Обґрунтування вибору засобів розробки

Для реалізації геолокаційної гри з використанням розширеної реальності на платформі iOS було вибрано ряд допоміжних програмних засобів, які забезпечують ефективний та надійний процес розробки. Основним засобом для розробки мобільного додатку є мова програмування Swift та фреймворк SwiftUI. Swift є сучасною мовою програмування, розробленою компанією Apple, яка забезпечує високу продуктивність та безпеку коду. SwiftUI, в свою чергу, пропонує декларативний підхід до побудови користувацького інтерфейсу, що значно спрощує розробку та підтримку додатку.

Розглядаючи можливі альтернативи для розробки користувацького інтерфейсу, можна було б використати UIKit, традиційний фреймворк для побудови UI на iOS. UIKit, який з'явився в iOS 2.0, забезпечує високий рівень контролю над інтерфейсними елементами, дозволяючи реалізувати складні взаємодії. Проте, цей фреймворк вимагає написання великої кількості коду, що ускладнює розробку та підтримку. UIKit використовує імперативний підхід, де розробник детально описує кожен крок створення інтерфейсу та його змін[11]. Натомість, SwiftUI, представлений у 2019 році, використовує декларативний підхід, що дозволяє розробникам описувати інтерфейс та його поведінку у відповідь на зміни стану за допомогою простих і зрозумілих конструкцій. Це значно скорочує кількість коду та спрощує процес розробки. SwiftUI автоматично керує оновленням інтерфейсу при зміні даних, що є критичним для AR-додатків, де швидкість та динамічність оновлень грають ключову роль. Для інтеграції з ARKit, SwiftUI надає набір готових компонентів, які легко

використовувати для створення AR-сцен та взаємодії з віртуальними об'єктами[13]. Це дозволяє зосередитись на логіці додатку, а не на рутинних завданнях управління інтерфейсом. SwiftUI також забезпечує кращу продуктивність завдяки оптимізації, яку Apple впроваджує у своїх фреймворках, що важливо для ресурсоємних задач, пов'язаних з AR. Крім того, SwiftUI підтримує багатоплатформну розробку, що дозволяє створювати інтерфейси для iOS, macOS, watchOS та tvOS, використовуючи один і той самий код. Це забезпечує консистентний користувацький досвід на різних пристроях Apple та спрощує підтримку додатку. Таким чином, для розробки AR-додатку на iOS, SwiftUI є більш ефективним вибором порівняно з UIKit. Він спрощує розробку завдяки декларативному підходу, забезпечує швидке та динамічне оновлення інтерфейсу, інтегрується з ARKit та підтримує багатоплатформність.

Для роботи з розширеною реальністю використовується фреймворк ARKit, який надає широкий спектр інструментів для створення AR-додатків. Альтернативами могли б бути сторонні рішення, такі як Vuforia або ARCore від Google. Vuforia, як один з найстаріших та найбільш поширених фреймворків для доповненої реальності, пропонує широкий спектр функцій, таких як розпізнавання зображень, об'єктів та маркерів[10]. Він підтримує різні платформи, включаючи iOS та Android, що забезпечує кросплатформність. Однак, для інтеграції з платформами iOS та Android, Vuforia часто потребує додаткових зусиль для налаштування та оптимізації, що може ускладнити розробку. ARCore, розроблений Google, спеціально оптимізований для пристроїв на базі Android. Він забезпечує високоякісне розпізнавання поверхонь, оцінку освітлення та можливості трекінгу руху. Хоча ARCore також підтримує iOS, його інтеграція з цією платформою може бути менш ефективною порівняно з нативними рішеннями. ARCore більше орієнтований на Android, що може призвести до обмеженої функціональності та продуктивності на iOS-пристроях. ARKit, розроблений Apple, є нативним фреймворком для iOS, що забезпечує високу продуктивність та ефективність. ARKit пропонує розширені можливості трекінгу руху, розпізнавання облич та об'єктів, а також інтеграцію з іншими

фреймворками Apple, такими як CoreML для машинного навчання[10]. Завдяки тісній інтеграції з апаратним забезпеченням iOS-пристроїв, ARKit забезпечує оптимальну продуктивність та низьке споживання ресурсів, що є критичним для AR-додатків. ARKit також підтримує новітні технології, такі як LiDAR, що дозволяє створювати більш точні та детальні 3D-моделі оточення[14]. Це забезпечує високу реалістичність та інтерактивність AR-додатків. Крім того, ARKit надає розробникам прості у використанні API та інструменти для швидкого створення та тестування AR-сцен, що значно спрощує процес розробки.

Для визначення місцезнаходження користувача використовується фреймворк CoreLocation, який забезпечує точні та надійні дані про геолокацію. Альтернативою могло б бути використання сторонніх SDK для геолокації, таких як Mapbox або Google Maps SDK[10]. Однак, CoreLocation інтегрований в iOS, що забезпечує максимальну сумісність та продуктивність, а також спрощує роботу з геолокаційними даними[6]. З іншого боку, Mapbox та Google Maps SDK пропонують розширені можливості для роботи з картографічними даними та геолокацією. Mapbox SDK надає потужні інструменти для кастомізації карт, підтримує 3D-візуалізацію та інтеграцію з AR. Він дозволяє створювати інтерактивні карти з високою деталізацією та кастомізацією[18]. Google Maps SDK, як відомий та широко використовуваний сервіс, пропонує доступ до великої бази даних карт та місць, що забезпечує високу точність та актуальність даних. Він також підтримує функції для роботи з картами, маршрутами та геокодингом[10]. Однак, використання сторонніх SDK може призводити до збільшення складності інтеграції та можливих проблем сумісності з іншими компонентами iOS. Крім того, сторонні SDK часто вимагають додаткових дозволів та налаштувань, що може ускладнити процес розробки та збільшити час на тестування.

На серверній стороні було обрано фреймворк Vapor, що працює з мовою програмування Swift. Vapor забезпечує високу продуктивність та масштабованість завдяки асинхронній обробці запитів і можливості писати

серверний код на тій самій мові, що і клієнтський додаток[20]. Альтернативами могли б бути середовища виконання, такі як Node.js з Express.js або Python з Flask чи Django. Node.js, завдяки своїй події-орієнтованій архітектурі, забезпечує високу продуктивність та масштабованість, особливо для додатків з великою кількістю паралельних запитів. Express.js, популярний фреймворк для Node.js, надає прості та гнучкі засоби для створення серверних додатків. Однак, використання Node.js потребує знань JavaScript, що вимагає від команди розробників володіння ще однією мовою програмування. Python з фреймворками Flask або Django також є популярним вибором для серверних додатків. Flask, як мікрофреймворк, забезпечує легкість та гнучкість у розробці, дозволяючи розробникам швидко створювати прототипи і малі додатки. Django, навпаки, є повноцінним фреймворком з багатьма вбудованими функціями, такими як аутентифікація, адміністрування та ORM. Проте, обидва ці фреймворки потребують знань мови програмування Python, що знову ж таки збільшує складність проекту при використанні різних мов для клієнтської та серверної частин[8].

Для зберігання даних у розробці AR-додатку було обрано реляційну базу даних MySQL та нереляційну базу даних MongoDB. MySQL забезпечує надійне та ефективне зберігання структурованих даних, підтримуючи складні SQL-запити та транзакції. Ця база даних є однією з найбільш поширених і широко підтримуваних, що забезпечує простоту інтеграції та використання у різних середовищах. Альтернативою могла б бути база даних PostgreSQL, яка також пропонує багаті функціональні можливості, включаючи підтримку складних типів даних та розширені можливості роботи з транзакціями. PostgreSQL відомий своєю високою продуктивністю та надійністю, однак MySQL часто вибирається через її широку популярність, велику кількість документації та підтримки, що полегшує навчання та розробку[1].

MongoDB була обрана для зберігання неструктурованих даних завдяки своїй гнучкості та масштабованості. Ця база даних підтримує документо-орієнтовану модель даних, що дозволяє ефективно зберігати та обробляти великі

обсяги неструктурованої інформації. Альтернативами могли б бути CouchDB або Firebase Firestore. CouchDB також є документо-орієнтованою базою даних, яка забезпечує високу доступність та масштабованість, однак MongoDB пропонує кращу інтеграцію з фреймворком Vapor та має більш зрозумілу модель даних для розробників, що працюють з JSON. Firebase Firestore від Google надає потужні можливості для масштабованого зберігання даних з інтеграцією у реальному часі, але MongoDB часто вибирається через свою гнучкість та можливість ефективно працювати з великими обсягами даних у різних форматах[9]. Усі сторонні бібліотеки для реалізації програмного забезпечення наведено у таблиці 3.1

Таблиця 3.1 - Допоміжні бібліотеки

№	Назва	Призначення
1	Alamofire	Бібліотека для роботи з HTTP-запитами в Swift
2	ARKit	Фреймворк для створення додатків з розширеною реальністю.
3	CoreLocation	Фреймворк для визначення геолокації користувача.
4	Vapor	Фреймворк для розробки серверної частини додатку на Swift.
5	SwiftyJSON	Бібліотека для зручної роботи з JSON в Swift
6	SnapKit	Бібліотека для створення автолейаутів за допомогою коду
7	MySQL	Реляційна база даних для зберігання структурованих даних
8	MongoDB	Нереляційна база даних для зберігання неструктурованих даних

3.3 Конструювання програмного забезпечення

Діаграма класів для геолокаційної гри з використанням розширеної реальності на платформі iOS включає основні компоненти системи, які

взаємодіють між собою для забезпечення повноцінного функціонування додатку. Структурна схема класів програми наведена у графічних матеріалах.

Основними класами, які виділені на діаграмі, є User, Achievement, Reward, Target, Game, Location, ARView, Server та Database. Клас User відповідає за збереження інформації про користувачів, таких як ідентифікатор, ім'я користувача, електронну пошту та пароль. Він також має зв'язки з іншими класами, такими як Achievement та Reward, що дозволяє користувачу отримувати нагороди та досягнення з подальшим використанням.

Клас Achievement зберігає інформацію про досягнення, включаючи ідентифікатор, назву, опис та дату досягнення. Зв'язок між класом User та Achievement реалізований як "1 до N", що означає, що один користувач може мати багато досягнень, а кожне досягнення належить одному користувачу[16]. Це дозволяє відстежувати досягнення кожного користувача, забезпечуючи можливість перегляду і управління ними. Крім того, клас Achievement зв'язується з класом Location, що дозволяє відстежувати, в яких місцях були отримані досягнення, додаючи контекст до досягнень користувачів і дозволяючи аналізувати, які локації є найбільш популярними. Безпосередньо нагороди зберігаються у класі Reward, який містить атрибути для збереження інформації про нагороди, такі як ідентифікатор, назва, опис та QR-код для викупу нагороди. Зв'язок між Reward та User також реалізований як "1 до N", що означає, що один користувач може отримати багато нагород, а кожна нагорода належить одному користувачу. Це дає можливість управління нагородами для кожного користувача, зокрема, дозволяючи їм викупляти нагороди, використовуючи QR-код. Зв'язок між Reward та Location реалізований як "1 до N", що дозволяє визначати, в яких локаціях можуть бути отримані нагороди, додаючи контекст до системи лояльності та забезпечуючи можливість аналізу популярності різних локацій.

Клас Target відповідає за віртуальні об'єкти, які користувачі можуть знаходити та взаємодіяти з ними у грі. Він містить інформацію про ідентифікатор та статус зібраного об'єкта, а також зв'язок з класом Location для визначення

місцезнаходження кожної мети. Клас Game зберігає інформацію про ігри, включаючи ідентифікатор гри та зв'язок з Target та Location, що дозволяє визначати, які цілі доступні в кожній грі та де вони розташовані. Клас Game зв'язаний з класом Location як "N до 1", що означає, що багато ігор можуть бути пов'язані з однією локацією, дозволяючи централізовано керувати місцезнаходженнями ігор.

Клас Location зберігає інформацію про локації, де розташовані віртуальні об'єкти та де можуть бути отримані нагороди. Він містить атрибути для збереження назви та координат локації, а також зв'язки з класами Reward, Achievement, Target та Game.

Клас ARView відповідає за відображення віртуальних об'єктів у доповненій реальності. Він містить атрибути для збереження AR-сесії та виду сцени, а також методи для запуску сесії, відображення цілей та оновлення сцени.

Клас Server відповідає за взаємодію з сервером і містить базову URL адресу сервера. Сервер може перевіряти валідність, відправляти запити та отримувати відповіді. Зв'язок між класом Server та класом Database має тип "один до одного", оскільки один сервер взаємодіє з однією базою даних. Клас Database відповідає за зберігання даних і містить підключення до бази даних. База даних може зберігати, завантажувати та оновлювати дані.

Для забезпечення структурування даних та уникнення аномалій при оновленні, було розроблено схему бази даних для геолокаційної гри з використанням розширеної реальності у третій нормальній формі (3НФ). 3НФ забезпечує гнучкість і масштабованість бази даних. Оскільки дані розділені на логічно пов'язані таблиці, можна легко додавати нові атрибути або змінювати існуючі без значного впливу на всю структуру бази даних[4]. Схема включає таблиці для користувачів, досягнень, ігор, цілей та зв'язків між ними. Схему бази даних зображено в графічних матеріалах. Опис таблиць бази даних наведено у таблицях 3.2 - 3.11.

Таблиця 3.2 - Опис таблиці User

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор користувача (PK)
username	varchar(45)	Ім'я користувача
email	varchar(45)	Електронна пошта
password	varchar(45)	Пароль користувача

Таблиця 3.3 - Опис таблиці Achievement

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор досягнення (PK)
location_id	binary(16)	Ідентифікатор локації (FK)
title	varchar(45)	Назва досягнення
description	varchar(128)	Опис досягнення
date_achieved	date	Дата досягнення

Таблиця 3.4 - Опис таблиці Reward

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор нагороди (PK)
location_id	binary(16)	Ідентифікатор локації (FK)
title	varchar(45)	Назва нагороди
description	varchar(128)	Опис нагороди
qr_code	varchar(128)	QR-код для отримання нагороди

Таблиця 3.5 - Опис таблиці Location

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор локації (PK)

Продовження таблиці 3.5

Назва поля	Тип даних	Опис
name	varchar(45)	Назва локації
coordinates	varchar(45)	Координати локації

Таблиця 3.6 - Опис таблиці Game

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор гри (PK)
location_id	binary(16)	Ідентифікатор локації (FK)

Таблиця 3.7 - Опис таблиці Target

Назва поля	Тип даних	Опис
id	binary(16)	Унікальний ідентифікатор цілі (PK)
location_id	binary(16)	Ідентифікатор локації (FK)
is_collected	bool	Статус зібрання цілі

Таблиця 3.8 - Опис таблиці User_Achievement

Назва поля	Тип даних	Опис
user_id	binary(16)	Ідентифікатор користувача (FK)
achievement_id	binary(16)	Ідентифікатор досягнення (FK)

Таблиця 3.9 - Опис таблиці User_Reward

Назва поля	Тип даних	Опис
user_id	binary(16)	Ідентифікатор користувача (FK)
reward_id	binary(16)	Ідентифікатор нагороди (FK)

Таблиця 3.10 - Опис таблиці User_Game

Назва поля	Тип даних	Опис
user_id	binary(16)	Ідентифікатор користувача (FK)
game_id	binary(16)	Ідентифікатор гри (FK)

Таблиця 3.11 - Опис таблиці Game_Target

Назва поля	Тип даних	Опис
game_id	binary(16)	Ідентифікатор гри (FK)
target_id	binary(16)	Ідентифікатор цілі (FK)

Виходячи з функціональних вимог до програмного забезпечення було розроблено 12 екранів для мобільного застосунку (табл. 3.12)

Таблиця 3.12 - Екрани мобільного застосунку

Назва	Призначення	Елементи
RegistrationView	Реєстрація	Поля введення (ім'я, email, пароль), кнопка «Зареєструватися»
LoginView	Авторизація	Поля введення (логін, пароль), кнопка «Увійти»
ChangePasswordView	Зміна паролю користувача	Поле введення email, поле введення коду, новий пароль, кнопка «Змінити пароль»
HomeView	Головний екран навігації	Кнопки навігації до інших екранів (почати гру, переглянути досягнення, налаштування)
GameView	Екран гри	Карта з поточним місцезнаходженням, список доступних цілей, кнопка «Почати гру»
ARInteractionView	Взаємодія з віртуальними об'єктами	AR-сцена, відображення віртуальних об'єктів, кнопка взаємодії
AchievementsView	Досягнення користувача	Список досягнень, кнопка повернення на головний екран
SettingsView	Налаштування	Поля введення для зміни налаштувань, кнопка збереження
RewardsView	Нагороди користувача	Список нагород, кнопка показу QR-коду нагороди

Продовження таблиці 3.12

Назва	Призначення	Елементи
LocationAdditionView	Панель додавання локацій	Поля введення (назва, координати), кнопка "Додати локацію"
TargetAdditionView	Панель додавання цілей	Поля введення (назва, координати), кнопка "Додати ціль"
RewardAdditionView	Панель додавання нагород	Поля введення (назва, опис, QR-код), кнопка "Додати нагороду"

Кожен екран реалізований з використанням інструментів і бібліотек Swift і SwiftUI. Екран `RegistrationView` забезпечує можливість реєстрації нового користувача. Для цього використовуються поля введення для імені, email та паролю, які реалізовані за допомогою `TextField` та `SecureField`. Кнопка "Зареєструватися" виконує перевірку введених даних та відправляє їх на сервер для створення нового користувача, використовуючи бібліотеку `Alamofire` для HTTP-запитів. Валідація даних включає перевірку на пусті поля та коректність введення email.

Екран `LoginView` забезпечує вхід користувача в систему. Він складається з полів введення для логіна та паролю, реалізованих за допомогою `TextField` та `SecureField`. Кнопка "Увійти" відправляє HTTP-запит на сервер для перевірки даних авторизації. У разі успішної авторизації користувача перенаправляють на `HomeView`, інакше відображається повідомлення про помилку.

Екран `ChangePasswordView` дозволяє користувачу змінити пароль. Користувач вводить свою email адресу для отримання коду підтвердження, після чого вводить новий пароль та код підтвердження. Поля введення реалізовані за допомогою `TextField` та `SecureField`. Кнопка "Змінити пароль" відправляє запит на сервер для перевірки коду та оновлення паролю. Валідація включає перевірку на відповідність нового паролю та коду підтвердження.

HomeView відображає основні функції гри і складається з кнопок для початку гри, перегляду досягнень та налаштувань. Використання NavigationLink дозволяє перемикатися між цими екранами. Кнопка "Почати гру" перенаправляє на GameView, кнопка "Переглянути досягнення" веде до AchievementsView, а кнопка "Налаштування" – до SettingsView.

Екран GameView відповідає за запуск гри та визначення місцезнаходження користувача. Кнопка "Запитати доступ до геолокації" викликає запит на доступ до геолокації за допомогою CoreLocation. Після отримання дозволу, на карті, реалізованій через MapKit, відображаються цілі гри. ARInteractionView відображає віртуальні об'єкти в розширеній реальності за допомогою ARKit. Користувач бачить об'єкти через камеру свого пристрою, взаємодія з об'єктами відбувається за допомогою жестів. Кнопка "Зібрати об'єкт" викликає функцію збирання об'єкту, оновлюючи дані на сервері та в базі даних.

Екран AchievementsView відображає досягнення користувача у вигляді списку, реалізованого через List. Кожне досягнення містить деталі, такі як назва, опис та дата отримання. Дані завантажуються з сервера за допомогою Alamofire та відображаються за допомогою SwiftUI.

SettingsView дозволяє користувачу налаштувати параметри гри. На цьому екрані реалізовані перемикачі (Toggle) для увімкнення або вимкнення звуків, сповіщень та зміни інтерфейсу. Налаштування зберігаються локально за допомогою UserDefaults, забезпечуючи збереження змін навіть після закриття додатку[25].

RewardsView відображає нагороди користувача, які він отримав за успішне виконання завдань у грі. Список нагород реалізований через List, кожна нагорода містить інформацію про назву, опис та QR-код для активації в реальному закладі. Користувач може показати QR-код у закладі для отримання знижки або бонусу.

LocationAdditionView надає адміністраторам інтерфейс для додавання нових локацій в гру. Поля введення для назви та координат локації реалізовані за допомогою TextField. Кнопка "Додати локацію" відправляє дані на сервер для збереження нової локації. TargetAdditionView дозволяє адміністраторам

додавати нові цілі для гри. Поля введення для назви та координат цілі реалізовані за допомогою TextField. Кнопка "Додати ціль" відправляє дані на сервер для збереження нової цілі. RewardAdditionView забезпечує адміністраторам можливість додавати нові нагороди. Поля введення для назви, опису та QR-коду реалізовані за допомогою TextField. Кнопка "Додати нагороду" відправляє дані на сервер для збереження нової нагороди.

На рахунок використаних алгоритмів, то задля збільшої точності геолокації(що є запорукою зручного геймплею) використано алгоритм Калмана, схема якого зображена на рисунку 3.5.

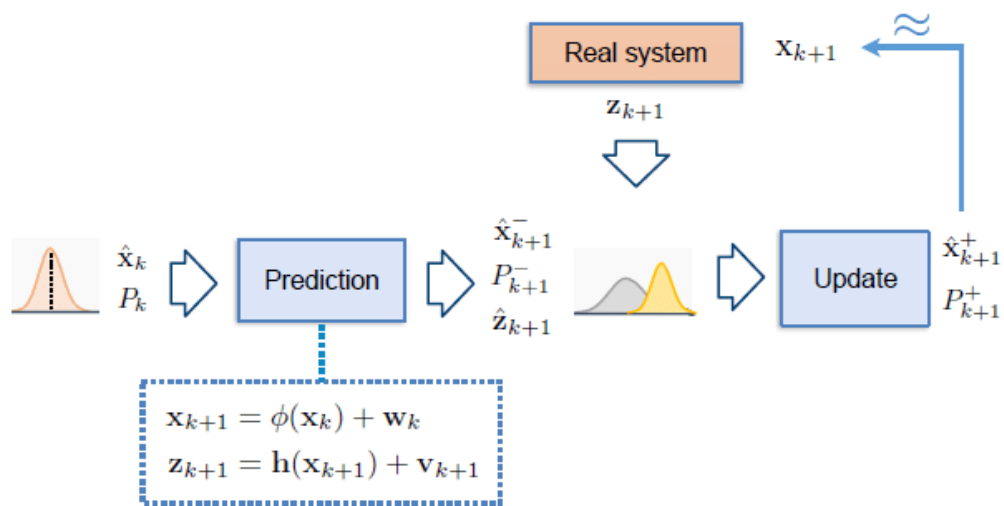


Рисунок 3.5 - Схема роботи алгоритму Калмана[17]

Алгоритм складається з двох основних етапів: передбачення та оновлення. На етапі передбачення алгоритм використовує модель динаміки системи для прогнозування майбутнього стану об'єкта. Це включає обчислення прогнозованого стану та коваріаційної матриці, яка представляє невизначеність цього прогнозу. Модель динаміки системи включає як детерміновані, так і стохастичні компоненти, що дозволяє враховувати випадкові зміни та шум у системі[5]. На етапі оновлення алгоритм об'єднує прогнозовані значення з новими вимірюваннями, отриманими з реальної системи. Цей процес включає обчислення залишкової похибки, яка є різницею між реальними вимірюваннями та прогнозованими значеннями. Використовуючи цю залишкову похибку, алгоритм Калмана коригує прогнозовані значення, щоб отримати оновлену

оцінку стану об'єкта. Вага, яка надається новим вимірюванням у порівнянні з прогнозом, визначається за допомогою коефіцієнтів Калмана, які розраховуються на основі ковариаційних матриць прогнозу та вимірювань. У геолокаційному додатку застосування алгоритму Калмана включає кілька кроків. Спочатку визначається початковий стан та ковариаційна матриця. Потім, на кожному кроці оновлення, алгоритм використовує поточні вимірювання для коригування прогнозованого стану[22].

Опис програмного забезпечення, яке брало участь у розробці, наведено в таблиці 3.13

Таблиця 3.13 - Програмне забезпечення для розробки додатку

№	Назва	Опис
1	Xcode	Середовище розробки для створення мобільних застосунків на iOS, включаючи Swift та SwiftUI.
2	MySQL Workbench	Система керування базами даних, використовується для зберігання даних користувачів, ігор та цілей.
3	Postman	Програмне забезпечення для тестування REST API запитів, використовується для тестування серверної частини.
4	Docker	Платформа для контейнеризації, використовується для розгортання серверного оточення.

Текст коду програми наведено в окремому документі під назвою «Текст програми».

3.4 Аналіз безпеки даних

Однією з основних вразливостей є передача даних між клієнтською частиною додатку та сервером. Незахищені канали передачі даних можуть стати мішенню для атак типу "людина посередині" (Man-in-the-Middle), що може призвести до викрадення або зміни переданих даних. Для запобігання таких атак

необхідно використовувати захищені протоколи, такі як HTTPS, які забезпечують шифрування даних під час їх передачі[23].

Зберігання даних користувачів також потребує ретельного підходу до безпеки. База даних повинна бути захищена від несанкціонованого доступу за допомогою механізмів контролю доступу та шифрування даних на сервері. Важливо забезпечити регулярне оновлення системи безпеки бази даних та моніторинг підозрілої активності для своєчасного виявлення і реагування на потенційні загрози.

Використання сторонніх бібліотек і фреймворків теж може стати джерелом вразливостей. Необхідно регулярно перевіряти та оновлювати всі зовнішні залежності, щоб уникнути використання версій з відомими вразливостями. Використання інструментів для статичного аналізу коду, таких як SwiftLint, допомагає виявляти потенційні проблеми безпеки на ранніх етапах розробки. Натомість, архітектурні рішення, такі як використання моделі MVVM (Model-View-ViewModel), також сприяють підвищенню безпеки. Вони дозволяють розділити логіку бізнесу, представлення даних та їх обробку, що знижує ризик виникнення помилок і вразливостей. Такий підхід також сприяє легшій підтримці та тестуванню коду, що важливо для забезпечення безпеки в довгостроковій перспективі[8].

Висновки до розділу

У цьому розділі було детально розглянуто архітектурні та технічні аспекти розробки мобільного застосунку для геолокаційної гри з використанням розширеної реальності на платформі iOS. Базова архітектура додатку включає модель MVVM, що забезпечує гнучкість та масштабованість системи, а також спрощує розробку та підтримку. Описана архітектура бази даних відповідає третій нормальній формі (3НФ), забезпечуючи структуроване зберігання даних та мінімізацію дублювання інформації. Було розроблено та проаналізовано ключові екрани мобільного застосунку, включаючи RegistrationView, LoginView, ChangePasswordView, MainView, StartGameView, ARInteractionView,

AchievementsView та SettingsView. Кожен екран має чітко визначену функціональність та реалізований з використанням сучасних інструментів і бібліотек Swift та SwiftUI. Особлива увага приділялася забезпеченню зручності використання та інтерактивності інтерфейсу користувача. Для серверної частини було обрано фреймворк Vapor на мові програмування Swift, що забезпечує високу продуктивність та масштабованість завдяки асинхронній обробці запитів. Розглянуто та обґрунтовано вибір технологічного стека, включаючи MySQL для зберігання структурованих даних та MongoDB для неструктурованих, Alamofire для обробки HTTP-запитів, ARKit для реалізації функцій розширеної реальності та CoreLocation для роботи з геолокацією. Крім того, було проведено аналіз безпеки даних, враховуючи потенційні вразливості та заходи для їхньої мінімізації.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Аналіз якості ПЗ

Основним інструментом для забезпечення якості коду виступав SwiftLint, який забезпечує автоматизоване перевірку коду на відповідність стандартам стилю та виявлення можливих помилок. Використання SwiftLint дозволяє автоматизувати процес лінтингу, знижуючи ймовірність появи синтаксичних помилок та невідповідностей стилю коду, що є критичним для підтримання високої якості коду[21]. SwiftLint інтегрувався у процес розробки як частина системи безперервної інтеграції, що дозволило автоматично перевіряти код на кожному етапі його розробки та виявляти можливі помилки ще на ранніх стадіях. Це допомогло забезпечити дотримання стандартів кодування та зменшити кількість потенційних багів у кінцевому продукті. Результати роботи SwiftLint показали відсутність серйозних порушень, що свідчить про високу якість коду та відповідність стандартам програмування на Swift. Результати аналізу зображено на рисунках 4.1-4.2.

```
Done linting! Found 0 violations, 0 serious in 51 files.  
yaroslav@MBP-Berlinskij ~ %
```

Рисунок 4.1 - Результати SwiftLint для аналізу коду додатку

```
Total errors found: 0  
yaroslav@MBP-Berlinskij ~ %
```

Рисунок 4.2 - Результати SwiftLint для аналізу коду сервера

Отже, лінтер показав, що код може бути підтримуваним та не має помилок в стилістиці його написання.

Задля аналізу результатів впливу відпрацювання коду на саму систему, скористаємось Xcode Instruments та продемонструємо на рисунку 4.3.

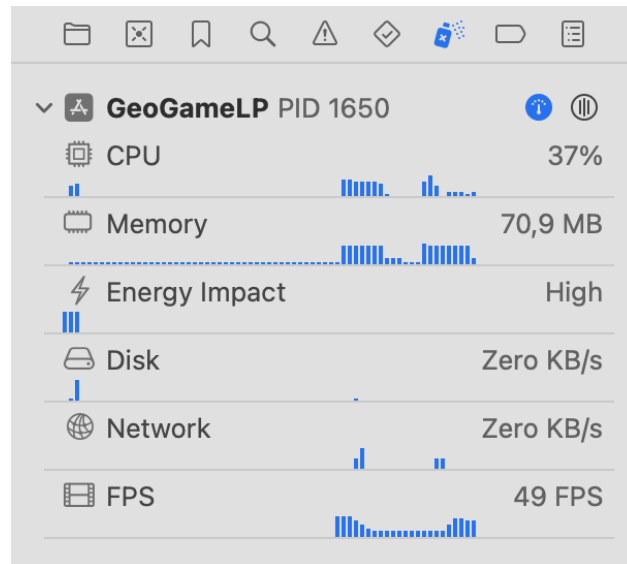


Рисунок 4.3 - Аналіз роботи програми

Ці інструменти дозволяють проводити детальний аналіз продуктивності мобільного додатку на рівні системи, визначаючи вплив окремих компонентів додатку на ресурси пристрою, такі як CPU, пам'ять, енергоспоживання, дискова активність, мережева активність та кількість кадрів в секунду (FPS). Згідно з результатами аналізу, під час виконання додатку витрати CPU в середньому 5-40%, що вказує на значну амплітуду використання процесорних ресурсів під час виконання основних функцій додатку, таких як обробка запитів до сервера та робота з розширеною реальністю, але це нормально в контексті роботи з AR-технологіями. Споживання пам'яті в середньому становило 52-100 MB, що є прийнятним для додатків з використанням AR технологій, які, між іншим, вимагають значних обсягів пам'яті для зберігання графічних даних та об'єктів, також це вписується до попередньо вироблених нефункціональних вимог. Енергоспоживання характеризувалося як високе, що зумовлено активним використанням CPU та інших ресурсів під час роботи додатку. Це може мати вплив на тривалість роботи батареї пристрою, але для AR-застосунків це, на жаль, вимушена практика[19]. Дискова активність та мережева активність були на мінімальному рівні, що вказує на те, що додаток не здійснював значних операцій зчитування або запису на диск та не генерував значного мережевого трафіку на момент тестування. Середня частота кадрів (FPS) становила 45-50 FPS, що є близьким до оптимального показника 60 FPS. Це свідчить про плавну

роботу додатку без помітних затримок або затримок у відображенні графічних елементів. Загалом, результати аналізу показують, що додаток працює стабільно та ефективно використовує ресурси системи.

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування». Було виконане мануальне тестування програмного забезпечення, опис відповідних тестів наведено у таблицях 4.1-4.15

Таблиця 4.1 - Реєстрація нового користувача

Початковий стан системи	Користувач знаходиться на екрані реєстрації
Вхідні дані	Ім'я, email, пароль
Опис проведення тесту	Користувач вводить ім'я, email та пароль і натискає кнопку "Зареєструватися"
Очікуваний результат	Новий користувач створений, перенаправлення на екран авторизації
Фактичний результат	Збігається з очікуваним

Таблиця 4.2 - Авторизація користувача

Початковий стан системи	Користувач знаходиться на екрані авторизації
Вхідні дані	Логін, пароль
Опис проведення тесту	Користувач вводить логін та пароль і натискає кнопку "Увійти"
Очікуваний результат	Користувач авторизований, перенаправлення на головний екран
Фактичний результат	Збігається з очікуваним

Таблиця 4.3 - Зміна пароля

Початковий стан системи	Користувач знаходиться на екрані зміни паролю
Вхідні дані	Email, код підтвердження, новий пароль
Опис проведення тесту	Користувач вводить email для отримання коду, потім вводить код підтвердження і новий пароль, натискає кнопку "Змінити пароль"
Очікуваний результат	Пароль успішно змінений, повідомлення про успішну зміну
Фактичний результат	Збігається з очікуванням

Таблиця 4.4 - Початок гри

Початковий стан системи	Користувач авторизований і знаходиться на головному екрані
Вхідні дані	-
Опис проведення тесту	Користувач натискає кнопку "Почати гру"
Очікуваний результат	Перенаправлення на екран гри з запитом на доступ до геолокації
Фактичний результат	Збігається з очікуванням

Таблиця 4.5 - Визначення геолокації

Початковий стан системи	Користувач на екрані гри, доступ до геолокації запитаний
Вхідні дані	-
Опис проведення тесту	Користувач дозволяє доступ до геолокації
Очікуваний результат	Відображення поточного місцезнаходження на карті
Фактичний результат	Збігається з очікуваним

Таблиця 4.6 - Відображення цілей на карті

Початковий стан системи	Користувач знаходиться на екрані гри, місцезнаходження визначене
Вхідні дані	-
Опис проведення тесту	Користувач переглядає карту з цілями
Очікуваний результат	Відображення цілей на карті
Фактичний результат	Збігається з очікуваним

Таблиця 4.7 - Взаємодія з віртуальними об'єктами

Початковий стан системи	Користувач знаходиться на екрані AR-взаємодії
Вхідні дані	-
Опис проведення тесту	Користувач взаємодіє з віртуальним об'єктом за допомогою жестів
Очікуваний результат	Відображення взаємодії, оновлення даних об'єкту
Фактичний результат	Збігається з очікуваним

Таблиця 4.8 - Отримання нагороди

Початковий стан системи	Користувач зібрав віртуальний об'єкт(або об'єкти)
Вхідні дані	-
Опис проведення тесту	Користувач взаємодіє з віртуальним об'єктом і отримує нагороду
Очікуваний результат	Відображення нагороди, можливість переглянути QR-код
Фактичний результат	Збігається з очікуваним

Таблиця 4.9 - Перегляд досягнень

Початковий стан системи	Користувач авторизований і знаходиться на головному екрані
Вхідні дані	-
Опис проведення тесту	Користувач натискає кнопку "Переглянути досягнення"
Очікуваний результат	Перенаправлення на екран досягнень, відображення списку досягнень
Фактичний результат	Збігається з очікуваним

Таблиця 4.10 - Зміна налаштувань

Початковий стан системи	Користувач авторизований і знаходиться на головному екрані
Вхідні дані	-
Опис проведення тесту	Користувач натискає кнопку "Налаштування", змінює параметри
Очікуваний результат	Зміни зберігаються, параметри оновлюються
Фактичний результат	Збігається з очікуваним

Таблиця 4.11 - Додавання локацій

Початковий стан системи	Адміністратор авторизований
Вхідні дані	Назва локації, координати
Опис проведення тесту	Адміністратор вводить дані нової локації і натискає кнопку "Додати"
Очікуваний результат	Нова локація додається в систему
Фактичний результат	Збігається з очікуванням

Таблиця 4.12 - Додавання віртуальних об'єктів

Початковий стан системи	Адміністратор авторизований
Вхідні дані	Назва об'єкта, локація, координати
Опис проведення тесту	Адміністратор вводить дані нового об'єкта і натискає кнопку "Додати"
Очікуваний результат	Новий об'єкт додається в систему
Фактичний результат	Збігається з очікуванням

Таблиця 4.13 - Відображення QR-коду нагороди

Початковий стан системи	Користувач переглядає нагороди
Вхідні дані	-
Опис проведення тесту	Користувач обирає нагороду і натискає кнопку "Показати QR-код"
Очікуваний результат	Відображення QR-коду нагороди на екрані
Фактичний результат	Збігається з очікуваним

Таблиця 4.14 - Пошук об'єктів на карті

Початковий стан системи	Користувач на екрані мапи
Вхідні дані	-
Опис проведення тесту	Користувач використовує функцію пошуку для знаходження об'єктів
Очікуваний результат	Відображення результатів пошуку на мапі
Фактичний результат	Збігається з очікуваним

Таблиця 4.15 - Отримання оновлень про нові об'єкти

Початковий стан системи	Користувач авторизований і знаходиться на головному екрані
Вхідні дані	-
Опис проведення тесту	Користувач перевіряє наявність нових об'єктів
Очікуваний результат	Відображення нових об'єктів на головному екрані
Фактичний результат	Збігається з очікуванням

4.3 Опис контрольного прикладу

Запускаючи додаток вперше, користувач бачить екран реєстрації (рис. 4.4)

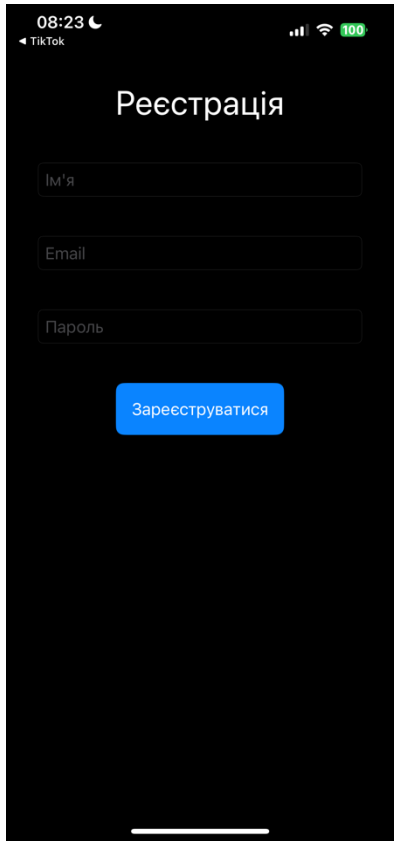


Рисунок 4.4 - Екран реєстрації

Користувач також може потрапити на екран авторизації, де матиме змогу увійти(як користувач), або ж за допомогою спеціальної кнопки увійти, як адміністратор (рис. 4.5)

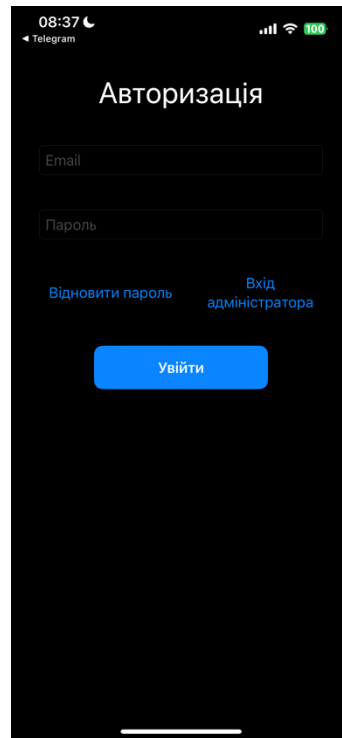


Рисунок 4.5 - Екран авторизації

Потрапивши в головне меню завдань, користувач бачить перелік локацій, тобто місць, де можна взяти участь в програмі лояльності з додатком (рис. 4.6)

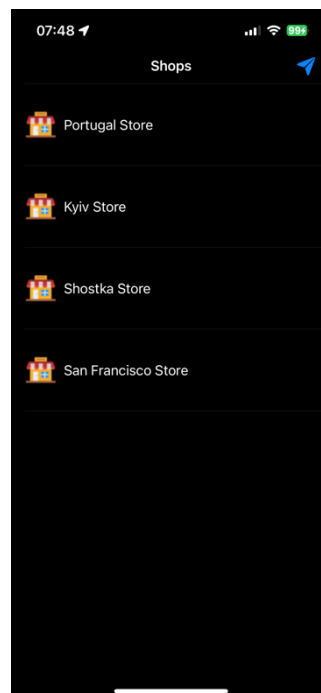


Рисунок 4.6 - Список локацій, де діють програми лояльності

Зручніше можна подивитися місця, які знаходяться в безпосередній близькості, за допомогою мапи, давши доступ на визначення місцязнаходження (рис. 4.7) та перейшовши на екран самої мапи (рис. 4.8), де можна обрати доступний магазин з програмою лояльності та взяти участь в квесті

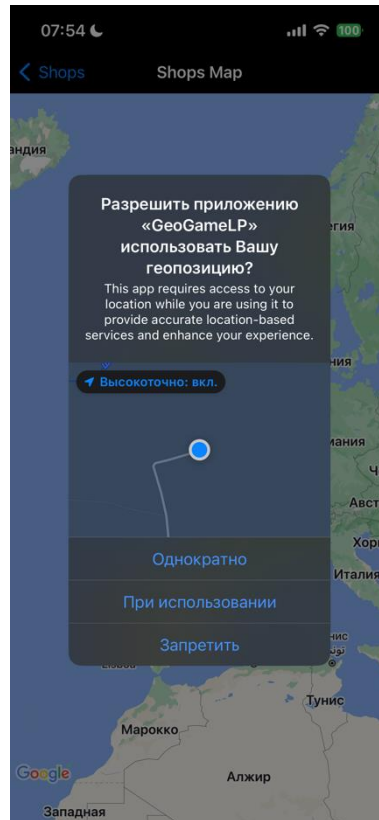


Рисунок 4.7 - Надання доступу до геоданих

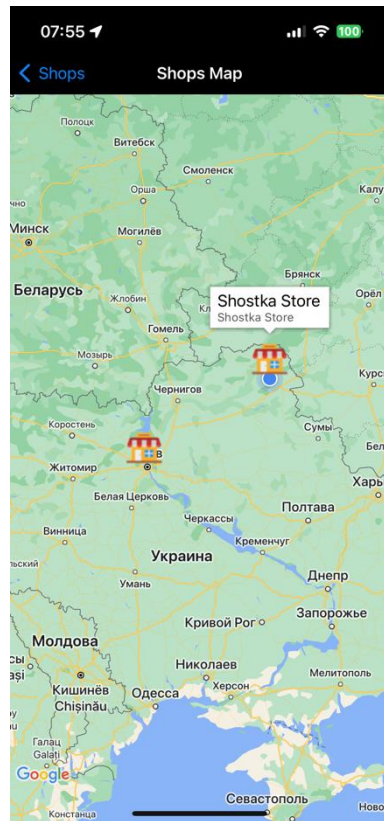


Рисунок 4.8 - Экран перегляду локацій з програмами лояльності

Обрати локацію з програмою лояльності можна також і повернувшись в головне меню із завданнями, де користувач отримає або сповіщення про заборону через недостатню близькість до місця проведення програми лояльності (рис. 4.9), або ж мати змогу взяти участь у квесті (рис. 4.10)

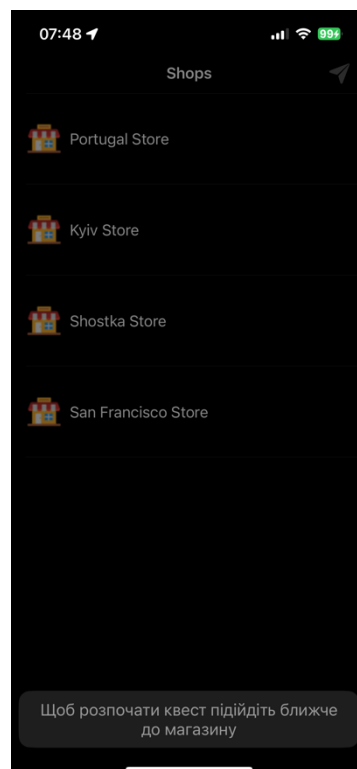


Рисунок 4.9 - Сповідання про заборону прийняти участь у програмі лояльності

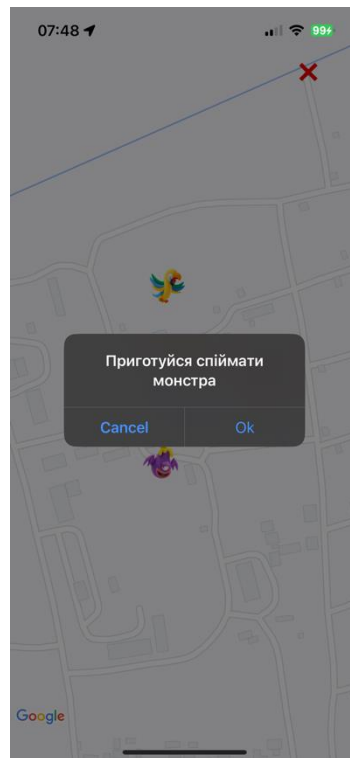


Рисунок 4.10 - Початок квесту програми лояльності

На екрані користувач бачить істот або предмети, які треба спіймати, щоби виконати квест. Якщо користувач перейде до місця знаходження зображуваного віртуального об'єкту, то він матиме змогу його спіймати через камеру свого смартфона (рис. 4.11), але спочатку об'єкт потрібно відшукати камерою



Рисунок 4.11 - Екран взаємодії з віртуальним об'єктом

Взаємодіяти з об'єктом можна завдяки жестам по екрану. Спіймати об'єкт можна шляхом натискання по ньому (рис. 4.12)



Рисунок 4.12 - Спійманий об'єкт

Якщо після успішного виконання квесту або квестів користувач отримує нагороду, то він може використати її в рамках програми лояльності через QR-код (рис. 4.13).



Рисунок 4.13 - Екран використання нагороди

Висновки до розділу

У цьому розділі було проведено ретельний аналіз якості та тестування програмного забезпечення, розробленого для мобільного AR-додатку. Одним з ключових інструментів, використаних для забезпечення високої якості коду, був SwiftLint, що автоматично мав виявити помилки стилю і синтаксису, контролюючи рівень стандартизації коду. Використання SwiftLint забезпечило дотримання кращих практик написання коду, що в свою чергу підвищило читабельність і підтримуваність проекту. Додатково було проведено детальне мануальне тестування всіх функціональних вимог додатку. Кожен компонент системи, включаючи реєстрацію, авторизацію, зміну паролю, запуск гри, взаємодію з віртуальними об'єктами, перегляд досягнень та налаштування, був протестований для перевірки коректності роботи. Тестові сценарії включали перевірку надійності введення даних, відповідність очікуваним результатам та поведінку системи при різних вхідних даних. Результати тестування продемонстрували досить високу стабільність та ефективність розробленого програмного забезпечення, як для AR-додатку. Під час тестування не було

виявлено критичних помилок, що підтверджує правильність вибраного підходу до розробки та тестування. Окрім цього, було проведено дослідження навантаження на систему під час роботи додатку, що показало прийнятний рівень споживання ресурсів, включаючи використання CPU, пам'яті та енергоефективність.

5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Процес розгортання програмного забезпечення, розробленого на Swift, відбувається через Xcode. Спочатку потрібно налаштувати Xcode проєкт для релізу, включаючи всі необхідні налаштування профілю підпису та сертифікатів. Це забезпечує аутентичність та безпеку додатку, дозволяючи уникнути потенційних проблем з перевіркою під час подання на App Store.

Процес налаштування проєкту для релізу через Xcode розпочинається з підготовки самого проєкту. Першим кроком є створення або вибір профілю підпису (Provisioning Profile) та сертифіката для підпису коду. Цей сертифікат має бути отриманий через Apple Developer Center, де потрібно створити новий профіль підпису, що включає ідентифікатор додатку (App ID) та пристрої, на яких додаток буде тестуватися. Наступним етапом є налаштування схем (Schemes) та конфігурацій збірки (Build Configurations) в Xcode. У проєкті необхідно мати окремі конфігурації для розробки (Debug) та релізу (Release). У налаштуваннях схеми важливо переконатися, що конфігурація збірки встановлена на Release для всіх відповідних таргетів. Це гарантує, що додаток буде компільовано з оптимізаціями, які підходять для продуктивного середовища, та включає в себе лише необхідні компоненти. Одним з важливих аспектів налаштування є конфігурація параметрів оптимізації та мінімізації розміру бінарного файлу[21]. У розділі "Build Settings" необхідно налаштувати параметри, такі як "Optimization Level" та "Dead Code Stripping", що дозволяють мінімізувати розмір додатку та покращити його продуктивність. Додатково, слід перевірити налаштування для зменшення розміру ресурсів, таких як зображення та звукові файли, використовуючи відповідні інструменти для компресії. Ще одним критичним етапом є налаштування параметрів безпеки, таких як включення ATS (App Transport Security) та налаштування захисту даних (Data Protection). Ці налаштування забезпечують відповідність додатку стандартам безпеки Apple, що є необхідною умовою для успішного проходження перевірки в App Store. Після завершення налаштування всіх параметрів необхідно виконати

повну перевірку проєкту на предмет наявності помилок та попереджень. Це включає запуск статичного аналізу коду, перевірку відповідності стандартам стилю коду, а також виконання всіх наявних тестів. Важливо переконатися, що всі тести пройдені успішно, а проєкт не містить критичних помилок, які можуть вплинути на його функціонування в продуктивному середовищі. Завершальним етапом є створення архіву додатку. У Xcode цей процес реалізується через меню **Product > Archive**, що генерує архівований файл (.ipa) готовий для подання на App Store. Архів також можна перевірити на наявність проблем за допомогою інструмента "Organizer", що забезпечує додаткову перевірку перед завантаженням[20].

Для завантаження додатку на App Store використовується App Store Connect – веб-портал, наданий Apple для розробників. Першим кроком є створення запису для нового додатку, де вказуються основні дані, такі як назва додатку, опис, ключові слова, категорії, політика конфіденційності та контактні дані. Важливо, щоб ця інформація була точною та відповідала вимогам Apple, оскільки вона впливає на процес перевірки та затвердження додатку. Після створення запису необхідно завантажити архівований файл .ipa до App Store Connect. Це можна зробити безпосередньо з Xcode, використовуючи вбудований інструмент для завантаження додатків, або через інструмент Application Loader. Важливо перевірити, чи файл завантажено успішно і чи не виникло помилок під час цього процесу. Після завантаження додатку починається процес перевірки, де Apple ретельно перевіряє додаток на відповідність усім вимогам та рекомендаціям. Це включає перевірку безпеки, продуктивності, стабільності та відповідності додатку вказаним описам і функціональним можливостям. Перевірка може тривати кілька днів, і у випадку виявлення проблем, розробнику надсилається звіт з детальним описом помилок і рекомендаціями щодо їх усунення[21].

5.2 Супровід програмного забезпечення

Інструкцію користувача розміщено в окремому документі.

Для кращого тестування або виправлення помилок додаток має постійно оновлюватись шляхом завантаження оновлень в AppStore. Також у зв'язку з тим, що AR-технології тільки перебувають на шляху свого становлення, варто слідкувати за тими рішеннями, які можуть покращити ефективність або в подальшому збільшити функціонал додатку.

Висновки до розділу

У цьому розділі було детально розглянуто процеси розгортання та супроводу програмного забезпечення, розробленого на платформі iOS з використанням Swift. Описано етапи налаштування проєкту для релізу через Xcode, включаючи підготовку профілів підпису та сертифікатів, налаштування схем та конфігурацій збірки, оптимізацію та мінімізацію розміру бінарного файлу, а також забезпечення відповідності стандартам безпеки Apple. Пояснено процедуру завантаження додатку на App Store через App Store Connect, зокрема створення запису додатку, завантаження архівованого файлу та процес перевірки Apple. Також висвітлено важливість постійного оновлення та супроводу програмного забезпечення, що включає виправлення помилок та впровадження нових функціональних можливостей. Окремо підкреслено значення моніторингу нових рішень у сфері AR-технологій для забезпечення подальшого розвитку та ефективності додатку.

ВИСНОВКИ

Отже, у результаті виконання дипломного проєкту було спроектовано та реалізовано мобільний додаток для геолокаційної гри з використанням розширеної реальності на платформі iOS. Цей додаток інтегрує сучасні технології AR та геолокації, що дозволяє користувачам взаємодіяти з віртуальними об'єктами в реальному світі. Одержані результати відповідають сучасному рівню наукових і технічних знань, враховуючи останні досягнення в області розробки мобільних додатків та використання AR технологій. У якості середовища розробки було обрано Xcode, що забезпечує високу продуктивність та зручність у написанні коду на мові Swift. Це середовище також надає всі необхідні інструменти для розгортання додатку на платформі iOS. Для серверної частини було використано фреймворк Vapor, який забезпечує високу продуктивність та безпеку обробки запитів. У якості БД використано реляційну базу даних MySQL для зберігання структурованих даних та нереляційну базу даних MongoDB для зберігання неструктурованих даних. Такий вибір забезпечує ефективне зберігання та обробку великих обсягів даних, необхідних для роботи додатку.

Ступінь впровадження результатів роботи є високим, оскільки створений додаток може бути легко інтегрований у різні галузі, такі як маркетинг, туризм та освіта, де використання AR технологій може значно покращити взаємодію користувачів з контентом. Можливі галузі або сфери використання результатів роботи включають розважальні додатки, освітні платформи та туристичні гідів.

Наукова, науково-технічна та соціально-економічна значущість роботи полягає у впровадженні нових технологій для створення інтерактивних та інноваційних додатків, що можуть сприяти розвитку відповідних галузей та підвищенню рівня технологічної обізнаності користувачів.

Доцільність продовження досліджень за відповідною тематикою включає вивчення нових можливостей ARKit, оптимізацію алгоритмів геолокації та інтеграцію додаткових функцій, що можуть підвищити зручність та ефективність використання додатку.

Стан вирішення усіх поставлених у дипломному проєкті задач є задовільним. Всі основні функціональні та нефункціональні вимоги були успішно реалізовані, що дозволило створити робочий прототип мобільного додатку, готовий до подальшого розвитку та вдосконалення.

ПЕРЕЛІК ПОСИЛАНЬ

1. Головачов І. А. Реляційні та нереляційні бази даних в бізнесі : thesis. 2018. URL: <https://er.knutd.edu.ua/handle/123456789/11102>(дата звернення: 12.05.2024).
2. Паршукова Л., Паршуков С. Доповнена реальність як спосіб урізноманітнення освітнього процесу. *Věda a perspektivy*. 2023. № 1(20). URL: [https://doi.org/10.52058/2695-1592-2023-1\(20\)-74-83](https://doi.org/10.52058/2695-1592-2023-1(20)-74-83)(дата звернення: 12.05.2024).
3. Системи Волинець В. О. Віртуальна, доповнена і змішана реальність: сутність понять та специфіка відповідних комп'ютерних систем. *Питання культурології*. 2021. № 37. С. 231–243. URL: <https://doi.org/10.31866/2410-1311.37.2021.237322>(дата звернення: 12.05.2024).
4. Ab M. MySQL administrator's guide and language reference (2nd edition). 2nd ed. MySQL Press, 2006. 888 p.
5. Alighanbari M., How J. P. Unbiased kalman consensus algorithm. *Journal of aerospace computing, information, and communication*. 2008. Vol. 5, no. 9. P. 298–311. URL: <https://doi.org/10.2514/1.34226>(date of access: 12.05.2024).
6. Core location | apple developer documentation. *Apple Developer Documentation*. URL: <https://developer.apple.com/documentation/corelocation>(date of access: 12.05.2024).
7. Dilek U., Erol M. Detecting position using ARKit. *Physics education*. 2018. Vol. 53, no. 2. P. 025011. URL: <https://doi.org/10.1088/1361-6552/aaa0e6>(date of access: 12.05.2024).
8. García R. IOS architecture patterns: MVP, MVVM, and VIPER in swift. Apress L. P., 2022.
9. Jain V., Upadhyay A. MongoDB and NoSQL Databases. *International journal of computer applications*. 2017. Vol. 167, no. 10. P. 16–20. URL: <https://doi.org/10.5120/ijca2017914385>(date of access: 12.05.2024).

10. Map application using augmented reality technology for smart phones / J. D. Jadhav et al. *International journal of science and engineering applications*. 2016. Vol. 5, no. 8. P. 421–424. URL: <https://doi.org/10.7753/ijsea0508.1003>(date of access: 20.05.2024).
11. Maskrey M., Wang W. Understanding ARKit. *Pro iPhone Development with Swift 4*. Berkeley, CA, 2018. P. 389–418. URL: https://doi.org/10.1007/978-1-4842-3381-8_13(date of access: 12.05.2024).
12. Nhan J. Advancing AR quick look for commerce on the web. *Mastering ARKit*. Berkeley, CA, 2022. P. 471–503. URL: https://doi.org/10.1007/978-1-4842-7836-9_24(date of access: 12.05.2024).
13. Nhan J. Introduction to arkit: under the hood and matrixes. *Mastering ARKit*. Berkeley, CA, 2022. P. 17–43. URL: https://doi.org/10.1007/978-1-4842-7836-9_2(date of access: 12.05.2024).
14. Nhan J. Reality composer: creating AR content. *Mastering ARKit*. Berkeley, CA, 2022. P. 309–335. URL: https://doi.org/10.1007/978-1-4842-7836-9_17(date of access: 12.05.2024).
15. Nhan J. Why augmented reality?. *Mastering ARKit*. Berkeley, CA, 2022. P. 1–15. URL: https://doi.org/10.1007/978-1-4842-7836-9_1(date of access: 12.05.2024).
16. Novitsky A. V. Extending UML specification for semantic modeling objects. *Problems in programming*. 2016. No. 2-3. P. 211–219. URL: <https://doi.org/10.15407/pp2016.02-03.211>(date of access: 12.05.2024).
17. On-board spacecraft relative pose estimation with high-order extended Kalman filter / F. Cavenago et al. *Acta astronautica*. 2019. Vol. 158. P. 55–67. URL: <https://doi.org/10.1016/j.actaastro.2018.11.020>(date of access: 20.05.2024).
18. Peddie J. Conclusions and future possibilities. *Augmented reality*. Cham, 2017. P. 297–302. URL: https://doi.org/10.1007/978-3-319-54502-8_10(date of access: 12.05.2024).

19. Peddie J. Overview of augmented reality system organization. *Augmented reality*. Cham, 2017. P. 53–58. URL: https://doi.org/10.1007/978-3-319-54502-8_4(date of access: 12.05.2024).
20. Radev T. Creating the project and app layout. *Create reminder app in xcode*. Berkeley, CA, 2023. URL: https://doi.org/10.1007/978-1-4842-9683-7_1(date of access: 12.05.2024).
21. Radev T. Testing the app and finishing the project. *Create reminder app in xcode*. Berkeley, CA, 2023. URL: https://doi.org/10.1007/978-1-4842-9683-7_5(date of access: 12.05.2024).
22. Shet K., Rao B. Modified fast Kalman algorithm. *IEEE transactions on acoustics, speech, and signal processing*. 1987. Vol. 35, no. 7. P. 1072–1076. URL: <https://doi.org/10.1109/tassp.1987.1165225>(date of access: 12.05.2024).
23. Vapor docs. *Vapor Docs*. URL: <https://docs.vapor.codes>(date of access: 01.06.2024).
24. Virtual indoor map with augmented reality navigation. *International research journal of modernization in engineering technology and science*. 2023. URL: <https://doi.org/10.56726/irjmets41126>(date of access: 20.05.2024).
25. Wang W. Understanding ARKit. *Pro iPhone Development with Swift 5*. Berkeley, CA, 2019. P. 519–553. URL: https://doi.org/10.1007/978-1-4842-4944-4_20(date of access: 12.05.2024).

ДОДАТОК А - ЗВІТ ПОДІБНОСТІ



Ім'я користувача:
Лісовиченко Олег Іванович

Дата перевірки:
12.06.2024 01:03:17 EEST

Дата звіту:
12.06.2024 07:05:22 EEST

ID перевірки:
1016348499

Тип перевірки:
Doc vs Internet + Library

ID користувача:
76913

Назва документа: ІП-01_Берлінський_ПЗ
Кількість сторінок: 73 Кількість слів: 12724 Кількість символів: 100941 Розмір файлу: 10.01 MB ID файлу: 1016151619

11.9%
Схожість

Найбільша схожість: 2.71% з джерелом з Бібліотеки (ID файлу: 1016131858)

4.36% Джерела з Інтернету	372	Сторінка 75
10.3% Джерела з Бібліотеки	505	Сторінка 78

0% Цитат

- Вилучення цитат вимкнене
- Вилучення списку бібліографічних посилань вимкнене

0%
Вилучень

Немає вилучених джерел

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ РЕАЛЬНОСТІ

Текст програми

КПІ.ПІ-0104.045490.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав БЕРЛІНСЬКИЙ

Київ – 2024

Файл RegistrationViewModel.swift

Реалізація функціональної задачі «Реєстрація користувачів»

```
class CharModel: ResponseModel {

    var name: String
    var imageURL: String

    private enum CodingKeys: String, CodingKey {
        case imageURL = "imageURL"
        case name = "name"
    }

    required init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.name = (try? values.decode(String.self, forKey: .name)) ?? ""
        self.imageURL = (try? values.decode(String.self, forKey: .imageURL)) ?? ""
    }
}

class CharsResponseModel: ResponseModel {

    var chars: [CharModel]

    private enum CodingKeys: String, CodingKey {
        case chars = "chars"
    }

    required init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.chars = (try? values.decode([CharModel].self, forKey: .chars)) ?? []
    }
}
```

Файл RegistrationView.swift

Реалізація функціональної задачі «Реєстрація користувачів»

```

import SwiftUI

struct RegistrationView: View {
    @ObservedObject var viewModel = RegistrationViewModel()

    var body: some View {
        VStack(spacing: 20) {
            TextField("Username", text: $viewModel.username)
                .textFieldStyle(RoundedBorderTextFieldStyle())
                .autocapitalization(.none)

            TextField("Email", text: $viewModel.email)
                .textFieldStyle(RoundedBorderTextFieldStyle())
                .autocapitalization(.none)
                .keyboardType(.emailAddress)

            SecureField("Password", text: $viewModel.password)
                .textFieldStyle(RoundedBorderTextFieldStyle())

            SecureField("Confirm Password", text: $viewModel.confirmPassword)
                .textFieldStyle(RoundedBorderTextFieldStyle())

            if !viewModel.errorMessage.isEmpty {
                Text(viewModel.errorMessage)
                    .foregroundColor(.red)
                    .multilineTextAlignment(.center)
            }

            Button(action: {
                viewModel.register()
            }) {
                Text("Register")
                    .padding()
                    .frame(maxWidth: .infinity)
                    .background(Color.blue)
                    .foregroundColor(.white)
                    .cornerRadius(10)
            }

            if viewModel.isRegistered {
                Text("Registration successful!")
                    .foregroundColor(.green)
            }
        }
    }
}

```

```

    }
    .padding()
}
}

```

Файл UserNetModel.swift

Реалізація функціональної задачі «Реєстрація користувачів»

```

import Foundation
import CoreLocation

class ShopModel: ResponseModel {

    var id: Int
    var lat: Double
    var lon: Double
    var name: String

    private enum CodingKeys: String, CodingKey {
        case id = "id"
        case lat = "lat"
        case lon = "lon"
        case name = "name"
    }

    required init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.id = (try? values.decode(Int.self, forKey: .id)) ?? -1
        self.name = (try? values.decode(String.self, forKey: .name)) ?? ""
        self.lat = (try? values.decode(Double.self, forKey: .lat)) ?? 0
        self.lon = (try? values.decode(Double.self, forKey: .lon)) ?? 0
    }

    func getLocation() -> CLLocation {
        return CLLocation(latitude: self.lat, longitude: self.lon)
    }
}

class ShopsResponseModel: ResponseModel {

    var shops: [ShopModel]
}

```

```

private enum CodingKeys: String, CodingKey {
    case shops = "shops"
}

required init(from decoder: Decoder) throws {
    let values = try decoder.container(keyedBy: CodingKeys.self)
    self.shops = (try? values.decode([ShopModel].self, forKey: .shops)) ??
[]
}

}

```

Файл RegistrationNetController.swift

Реалізація функціональної задачі «Реєстрація користувачів»

```

import Foundation

class NetworkError: Error {
    var statusCode: Int?

    init() {

    }
}

class ErrorModel: NetworkError, ResponseModel {

    var error: String?

    private enum CodingKeys: String, CodingKey {
        case error = "error"
    }

    required init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.error = try? values.decode(String.self, forKey: .error)
    }

}

```

Файл RegistrationNetDB.swift

Реалізація функціональної задачі «Реєстрація користувачів»

```

import Foundation

```

```

enum RequestHTTPMethod: String {
    case get = "GET"
    case post = "POST"
    case put = "PUT"
    case patch = "PATCH"
    case delete = "DELETE"
}

protocol RequestManager {
    init()
    func request<T: Decodable, E: Decodable & NetworkError>(
        requestModel: Encodable?,
        url: URL,
        type: RequestHTTPMethod,
        headers: [String : String]?,
        completion: @escaping (_ model: T?, _ backendError: E?, _ statusCode:
Int?) -> Void
    )

    func request<T: Decodable, E: Decodable & NetworkError>(
        requestModel: Encodable?,
        url: URL,
        type: RequestHTTPMethod,
        headers: [String : String]?,
        completion: @escaping (_ model: T?, _ backendError: E?) -> Void
    )
}

```

Файл LoginModel.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення паролю»

```

import Foundation
import Alamofire
import AlamofireNetworkActivityIndicator

class AlamofireRequest: RequestManager {

    internal func request<T: Decodable, E: Decodable & NetworkError>(
        requestModel: Encodable?,
        url: URL,
        type: RequestHTTPMethod,

```

```

        headers: [String : String]?,
        completion: @escaping (_ model: T?, _ backendError: E?) -> Void) {

            self.request(requestModel: requestModel, url: url, type: type, headers:
headers) { (response, error, _) in
                completion(response, error)
            }
        }

internal func request<T: Decodable, E: Decodable & NetworkError>(
    requestModel: Encodable?,
    url: URL,
    type: RequestHTTPMethod,
    headers: [String : String]?,
    completion: @escaping (_ model: T?, _ backendError: E?, _ statusCode:
Int?) -> Void
    ) {

        let headers = (headers ?? [:])

        let ahs = HTTPHeaders(headers)

        AF.request(url, method: type.alamofireHTTPMethod, parameters:
requestModel?.dictionary, encoding: JSONEncoding.default, headers:
ahs).responseData { (response) in

            switch response.result {
            case .success(let v):
                if ((response.response?.statusCode ?? 0) / 100 == 2) { //200,
201 ... 299
                    completion(try? JSONDecoder().decode(T.self, from: v),
nil, response.response?.statusCode)
                } else {
                    let error: E? = try? JSONDecoder().decode(E.self, from: v)
                    error?.statusCode = response.response?.statusCode
                    completion(nil, error, response.response?.statusCode)
                }
            case .failure(_):
                completion(nil, nil, response.response?.statusCode)
            }
        }
    }
}

```

```

    }

    required init() {
        NetworkActivityLogger.shared.level = .debug
        NetworkActivityLogger.shared.startLogging()
    }

}

extension Encodable {
    var dictionary: [String: Any]? {
        guard let data = try? JSONEncoder().encode(self) else { return nil }
        return (try? JSONSerialization.jsonObject(with: data, options:
.allowFragments)).flatMap { $0 as? [String: Any] }
    }
}

extension RequestHTTPMethod {
    var alamofireHTTPMethod: Alamofire.HTTPMethod {
        switch self {
            case .get:
                return .get
            case .post:
                return .post
            case .put:
                return .put
            case .patch:
                return .patch
            case .delete:
                return .delete
        }
    }
}

```

Файл LoginViewModel.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення паролю»

```

import Foundation

class RequestHelper {

```



```

static let shared: RequestHelper = RequestHelper()

private(set) var requestManager: RequestManager? = AlamofireRequest()

private init() {

}

func setupRequestManaget(requestManager: RequestManager) {
    self.requestManager = requestManager
}

}

protocol Request {
}

```

Файл PasswordResetViewModel.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення паролю»

```

import Foundation

protocol BaseUrl {
    var rawValue: String {get}
}

enum GLPurl: String, BaseUrl {
    case BASE_URL = "https://hcr-res.fral.cdn.digitaloceanspaces.com/kpi/"
}

protocol Endpoint {
    var rawValue: String {get}
}

enum GLPendpoint: String, Endpoint {
    case shops = "shops.json"
    case chars = "chars.json"
}

```

```

class URLBuilder {
    class func build(endpoint: Endpoint) -> URL {
        return URLBuilder.build(endpoint: endpoint.rawValue)
    }

    class func build(url: GLPurl = GLPurl.BASE_URL, endpoint: String) -> URL {
        let urlString = ((url.rawValue +
endpoint).addingPercentEncoding(withAllowedCharacters:
CharacterSet.urlQueryAllowed) ?? "")
        print("-----")
        print(urlString)
        return URL(string: urlString)!
    }
}

```

Файл LoginView.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення паролю»

```

import UIKit

extension CALayer {

    var center: CGPoint {
        get {
            return CGPointMake(CGRectGetMidX(self.frame),
CGRectGetMidY(self.frame))
        }

        set {
            self.frame.origin.x = newValue.x - (self.frame.size.width / 2)
            self.frame.origin.y = newValue.y - (self.frame.size.height / 2)
        }
    }

    var width: CGFloat {
        return self.bounds.width
    }

    var height: CGFloat {
        return self.bounds.height
    }
}

```

```

        var origin: CGPoint {
            return CGPoint(x: self.center.x - (self.width / 2), y: self.center.y -
(self.height / 2))
        }

    }

extension CALayer {

    func springToMiddle(withDuration duration: CFTimeInterval, damping:
CGFloat, inView view: UIView) {
        let springX = CASpringAnimation(keyPath: "position.x")
        springX.damping = damping
        springX.fromValue = self.center.x
        springX.toValue = CGRectGetMidX(view.frame)
        springX.duration = duration
        self.add(springX, forKey: nil)

        let springY = CASpringAnimation(keyPath: "position.y")
        springY.damping = damping
        springY.fromValue = self.center.y
        springY.toValue = CGRectGetMidY(view.frame)
        springY.duration = duration
        self.add(springY, forKey: nil)
    }

    func centerInView(view: UIView) {
        self.center = CGPoint(x: CGRectGetMidX(view.frame), y:
CGRectGetMidY(view.frame))
    }

    func fadeOutWithDuration(duration: CFTimeInterval) {
        let fadeOut = CABasicAnimation(keyPath: "opacity")
        fadeOut.delegate = self
        fadeOut.duration = duration
        fadeOut.autoreverses = false
        fadeOut.fromValue = 1.0
        fadeOut.toValue = 0.6
        fadeOut.fillMode = CAMediaTimingFillMode.both
        fadeOut.isRemovedOnCompletion = false
        self.add(fadeOut, forKey: "myanimation")
    }
}

```

```

}

extension CALayer: CAAAnimationDelegate {

}

```

Файл PasswordResetView.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення паролю»

```

import UIKit
import GoogleMaps
import CoreLocation
import SVProgressHUD
import EasyAleert

class GameMapViewController: UIViewController {

    @IBOutlet weak var closeButton: UIButton!

    private var mapView: GMSMapView!

    private var locationManager = CLLocationManager()

    private var firstCentration: Bool = false

    private var shop: ShopModel!
    private var chars: [MapChar] = []

    class func create(shop: ShopModel) -> GameMapViewController {
        let vc = GameMapViewController()
        GameMapViewController.storyboardInst.instantiateViewController(withIdentifier: GameMapViewController.identifier) as! GameMapViewController
        vc.shop = shop
        return vc
    }

    override func viewDidLoad() {
        super.viewDidLoad()

        self.initMapView()
    }

```

```

self.locationManager.delegate = self

SVProgressHUD.show()
GetCharsRequest().request { chars, backendError in
    if let chars = chars {
        var newLocation = self.generateRandomLocation(center:
self.shop.getLocation(), radius: 100)
        chars.chars.forEach { char in

            var fl: Bool = true
            while(fl) {
                fl = false
                newLocation = self.generateRandomLocation(center:
self.shop.getLocation(), radius: 100)
                self.chars.forEach { char in
                    if (char.location.distance(from: newLocation) <
100) {

                        fl = true
                        return
                    }
                }
            }

            let mapChar = MapChar(char: char, location: newLocation)
            self.chars.append(mapChar)

            if let url = URL(string: char.imageUrl), let data = try?
Data(contentsOf: url) {
                mapChar.image = UIImage(data: data)

                mapChar.marker = GMSMarker()
                mapChar.marker?.icon =
mapChar.image?.imageWith(newSize: CGSize(width: 40, height: 40))
                mapChar.marker?.position = mapChar.location.coordinate
                mapChar.marker?.map = self.mapView
            }
        }
        SVProgressHUD.dismiss()
    }
}

```

```

        } else {
            SVProgressHUD.dismiss()
            self.dismiss(animated: true)
        }
    }
}

override func viewDidAppear(_ animated: Bool) {
    super.viewDidAppear(animated)
    self.locationManager.startUpdatingLocation()
}

override func viewWillDisappear(_ animated: Bool) {
    super.viewWillDisappear(animated)
    self.locationManager.stopUpdatingLocation()
}

private func initMapView() {
    let options = GMSMapViewOptions()
    if let shop = self.shop {
        options.camera = GMSCameraPosition.camera(withLatitude: shop.lat,
longitude: shop.lon, zoom: 6.0)
    }

    options.frame = self.view.frame

    self.mapView = GMSMapView(options: options)
    self.mapView.delegate = self
    self.view.insertSubview(self.mapView, at: 0)

    self.mapView.isMyLocationEnabled = true

    if let shop = self.shop {
        let shopMarker = GMSMarker()
        shopMarker.icon = UIImage(named: "shopIcon")
        shopMarker.position = CLLocationCoordinate2D(latitude: shop.lat,
longitude: shop.lon)
        shopMarker.map = self.mapView
    }
}

```

```

        override func viewWillTransition(to size: CGSize, with coordinator:
UIViewControllerTransitionCoordinator) {
            Timer.scheduledTimer(withTimeInterval: 0, repeats: false) { _ in
                self.mapView.frame = self.view.frame
            }
        }

        @IBAction func onClose(_ sender: Any) {
            self.dismiss(animated: true)
        }

        private func generateRandomLocation(center: CLLocation, radius: Double) ->
CLLocation {
            let radiusInMeters: Double = radius
            let earthRadius: Double = 6371000 // Earth's radius in meters

            // Convert radius from meters to radians
            let radiusInRadians = radiusInMeters / earthRadius

            // Generate two random numbers
            let u = Double.random(in: 0...1)
            let v = Double.random(in: 0...1)

            // Convert random numbers to polar coordinates
            let w = radiusInRadians * sqrt(u)
            let t = 2 * Double.pi * v

            // Convert center latitude and longitude to radians
            let centerLatRadians = center.coordinate.latitude.toRadians()
            let centerLonRadians = center.coordinate.longitude.toRadians()

            // Calculate new latitude in radians
            let newLatRadiansPart1 = sin(centerLatRadians) * cos(w)
            let newLatRadiansPart2 = cos(centerLatRadians) * sin(w) * cos(t)
            let newLatRadians = asin(newLatRadiansPart1 + newLatRadiansPart2)

            // Calculate new longitude in radians
            let newLonRadiansPart1 = sin(w) * sin(t)
            let newLonRadiansPart2 = cos(centerLatRadians) * cos(w)
            let newLonRadiansPart3 = sin(centerLatRadians) * newLonRadiansPart1
            let newLonRadians = centerLonRadians + atan2(newLonRadiansPart1,
newLonRadiansPart2 - newLonRadiansPart3)

```

```

        // Convert radians back to degrees
        let newLat = newLatRadians.toDegrees()
        let newLon = newLonRadians.toDegrees()

        return CLLocation(latitude: newLat, longitude: newLon)
    }
}

extension GameMapViewController: CLLocationManagerDelegate {
    func locationManager(_ manager: CLLocationManager, didUpdateLocations
locations: [CLLocation]) {
        if let location = locations.last {
            let camera = GMSCameraPosition.camera(withLatitude:
location.coordinate.latitude, longitude: location.coordinate.longitude, zoom:
17)

            if (!firstCentration) {
                self.mapView.animate(to: camera)
                self.firstCentration = true
            }

            self.chars.forEach { char in
                print(char.location.distance(from: location))
                if (char.location.distance(from: location) < 20) {
                    self.locationManager.stopUpdatingLocation()
                    EasyAlert(delegate: self).showConfirmationAlert(title:
"Приготуйся спіймати монстра") {
                        let gvc = GameViewController.create(treasure:
Treasure(name: char.char.name, image: char.image))
                        gvc.delegate = self
                        gvc.modalPresentationStyle = .fullScreen
                        self.present(gvc, animated: true)

                    } cancelCompletion: {
                        self.locationManager.startUpdatingLocation()
                    }

                }

            }

            return
        }
    }
}

```



```

    }

}

}

extension GameMapViewController: GMSMapViewDelegate {
    func mapView(_ mapView: GMSMapView, didTap marker: GMSMarker) -> Bool {
        self.locationManager(self.locationManager, didUpdateLocations:
[CLLocation(latitude: marker.position.latitude, longitude:
marker.position.longitude)])
        return true
    }
}

extension GameMapViewController: GameViewControllerDelegate {
    func charCathed(gvc: GameViewController, charName: String) {
        self.chars.first(where: {$0.char.name == charName})?.marker?.map = nil
        self.chars.removeAll(where: { $0.char.name == charName })
    }
}

extension GameMapViewController: EasyAlertDelegate {
    func show(_ alert: UIAlertController) {
        self.present(alert, animated: true, completion: nil)
    }
}

extension GameMapViewController: StoryboardInstantiable {
    static var storyboardInst: UIStoryboard {
        return UIStoryboard.main
    }
}

class MapChar {
    var char: CharModel
    var location: CLLocation
    var marker: GMSMarker?
    var image: UIImage?

```

```

        init(char: CharModel, location: CLLocation) {
            self.char = char
            self.location = location
        }
    }
}

```

Файл LoginNetController.swift

Реалізація функціональної задачі «Авторизація користувачів та відновлення пароллю»

```

import UIKit
import AVFoundation
import CoreMotion

final class GameViewController: UIViewController {

    let captureSession = AVCaptureSession()
    let motionManager = CMMotionManager()
    var previewLayer: AVCaptureVideoPreviewLayer!

    var treasure: Treasure!

    var foundImageView: UIImageView!
    var dismissButton: UIButton!

    var delegate: GameViewControllerDelegate?

    class func create(treasure: Treasure) -> GameViewController {
        let vc = GameViewController()
        vc.treasure = treasure
        return vc
    }

    var quaternionX: Double = 0.0 {
        didSet {
            if !foundTreasure { treasure.item.center.y = (CGFloat(quaternionX)
* view.bounds.size.width - 180) * 4.0 }
        }
    }

    var quaternionY: Double = 0.0 {

```

```

        didSet {
            if !foundTreasure { treasure.item.center.x = (CGFloat(quaternionY)
* view.bounds.size.height + 100) * 4.0 }
        }
    }

    var foundTreasure = false

    override func viewDidLoad() {
        super.viewDidLoad()
        view.backgroundColor = UIColor.black

        DispatchQueue.main.async {
            self.setupMainComponents()
        }
    }

    override func viewWillAppear(_ animated: Bool) {
        super.viewWillAppear(animated)
    }

    private func setupMainComponents() {
        setupCaptureCameraDevice()
        setupPreviewLayer()
        setupMotionManager()
        setupGestureRecognizer()
        setupDismissButton()
    }
}

extension GameViewController {
    private func setupDismissButton() {
        dismissButton = UIButton(type: .system)
        dismissButton.setTitle("✖", for: .normal)
        dismissButton.titleLabel?.font = UIFont.systemFont(ofSize: 25)
        dismissButton.setTitleColor(UIColor.red, for: .normal)
        dismissButton.addTarget(self, action: #selector(close), for:
        .touchUpInside)
        dismissButton.translatesAutoresizingMaskIntoConstraints = false
        dismissButton.alpha = 0.0
    }
}

```

```

        view.addSubview(dismissButton)
        dismissButton.bottomAnchor.constraint(equalTo: view.bottomAnchor,
constant: -14.0).isActive = true
        dismissButton.centerXAnchor.constraint(equalTo:
view.centerXAnchor).isActive = true
    }

    @objc func close() {
        self.dismiss(animated: true)
    }

    private func animateInDismissButton() {
        UIView.transition(with: self.dismissButton, duration: 2.5, options:
.transitionCrossDissolve) {
            self.dismissButton.alpha = 1.0
        }
    }
}

extension GameViewController {

    private func setupCaptureCameraDevice() {
        if
            let cameraDevice =
AVCaptureDevice.default(.builtInWideAngleCamera, for: .video, position:
.back),
            let cameraDeviceInput = try? AVCaptureDeviceInput(device:
cameraDevice) {

            if (captureSession.canAddInput(cameraDeviceInput)) {
                captureSession.addInput(cameraDeviceInput)
                DispatchQueue.global().async {
                    self.captureSession.startRunning()
                }
            }
        }
    }

    private func setupPreviewLayer() {
        previewLayer = AVCaptureVideoPreviewLayer(session: captureSession)
    }
}

```

```

        previewLayer.frame = view.bounds

        if treasure.image != nil {
            let height = treasure.image!.size.height
            let width = treasure.image!.size.height
            treasure.item.bounds = CGRectMake(100.0, 100.0, width, height)
            treasure.item.position = CGPointMake(view.bounds.size.height / 2,
view.bounds.size.width / 2)
            previewLayer.addSublayer(treasure.item)
            view.layer.addSublayer(previewLayer)
        }
    }

}

extension GameViewController {

    private func setupMotionManager() {

        if motionManager.isDeviceMotionAvailable &&
motionManager.isAccelerometerAvailable {
            motionManager.deviceMotionUpdateInterval = 2.0 / 60.0
            motionManager.startDeviceMotionUpdates(to:
OperationQueue.current!) { motion, error in
                if error != nil { print("wtf. \(String(describing: error))");
return }

                guard let motion = motion else { print("Couldn't unwrap
motion"); return }

                self.quaternionX = motion.attitude.quaternion.x
                self.quaternionY = motion.attitude.quaternion.y
            }
        }
    }

}

extension GameViewController {

    private func setupGestureRecognizer() {
        let gestureRecognizer = UITapGestureRecognizer(target: self, action:
#selector(self.viewTapped(_:)))
        gestureRecognizer.cancelsTouchesInView = false
    }
}

```

```

        view.addGestureRecognizer(gestureRecognizer)
    }

    @objc func viewTapped(_ gesture: UITapGestureRecognizer) {
        let location = gesture.location(in: view)

        let topLeftX = Int(treasure.item.origin.x)
        let topRightX = topLeftX + Int(treasure.item.width)
        let topLeftY = Int(treasure.item.origin.y)
        let bottomLeftY = topLeftY + Int(treasure.item.height)

        guard topLeftX < topRightX && topLeftX < bottomLeftY else { return }

        let xRange = topLeftX...topRightX
        let yRange = topLeftY...bottomLeftY

        checkForRange(xRange: xRange, yRange: yRange, withLocation: location)
    }

    private func checkForRange(xRange: ClosedRange<Int>, _ yRange:
ClosedRange<Int>, withLocation location: CGPoint) {
        guard foundTreasure == false else { return }

        let tapIsInRange = xRange.contains(Int(location.x)) &&
yRange.contains(Int(location.y))

        if tapIsInRange {

            foundTreasure = true
            motionManager.stopDeviceMotionUpdates()
            captureSession.stopRunning()

            treasure.item.springToMiddle(withDuration: 1.5, damping: 9,
inView: view)
            treasure.item.centerInView(view: view)

            previewLayer.fadeOutWithDuration(duration: 1.0)

            animateInTreasure()
            animateInDismissButton()
            displayNameOfTreasure()
            displayDiscoverLabel()
        }
    }

```

```

        self.delegate?.charCathed(gvc: self, charName: self.treasure.name)

    }

}

extension GameViewController {

    func animateInTreasure() {
        let frame = treasure.item.frame
        let image = treasure.image!
        foundImageView = UIImageView(image: image)
        foundImageView.alpha = 0.0
        foundImageView.frame = frame
        view.addSubview(foundImageView)

        UIView.animate(withDuration: 1.5, delay: 0.8, options: []) {
            self.foundImageView.alpha = 1.0
        }
    }

    func displayDiscoverLabel() {
        let label = UILabel(frame: CGRectZero)
        label.translatesAutoresizingMaskIntoConstraints = false
        label.font = UIFont(name: "AvenirNext-Regular", size: 30.0)
        label.text = "Caught!"
        label.numberOfLines = 1
        label.textAlignment = .center
        label.adjustsFontSizeToFitWidth = true
        label.textColor = UIColor.white
        label.alpha = 0.0

        view.addSubview(label)
        label.centerXAnchor.constraint(equalTo: view.centerXAnchor).isActive =
true
        label.topAnchor.constraint(equalTo: view.topAnchor, constant:
50.0).isActive = true
        label.leftAnchor.constraint(equalTo: view.leftAnchor, constant:
14.0).isActive = true
        label.rightAnchor.constraint(equalTo: view.rightAnchor, constant: -
14.0).isActive = true
    }
}

```

```

        label.center.x -= 800
        label.alpha = 1.0

        UIView.animate(withDuration: 1.5, delay: 0.5, usingSpringWithDamping:
0.8, initialSpringVelocity: 4.0, options: []) {
            label.center.x = self.view.center.x
        }

    }

    func displayNameOfTreasure() {
        let label = UILabel(frame: CGRectZero)
        label.translatesAutoresizingMaskIntoConstraints = false
        label.font = UIFont(name: "AvenirNext-Regular", size: 45.0)
        label.text = treasure.name
        label.numberOfLines = 1
        label.textAlignment = .center
        label.adjustsFontSizeToFitWidth = true
        label.textColor = UIColor.blue
        label.alpha = 0.0

        view.addSubview(label)

        label.centerXAnchor.constraint(equalTo:
foundImageView.centerXAnchor).isActive = true
        label.topAnchor.constraint(equalTo: foundImageView.bottomAnchor,
constant: 14.0).isActive = true
        label.centerYAnchor.constraint(equalTo:
foundImageView.centerYAnchor).isActive = false
        label.leftAnchor.constraint(equalTo: view.leftAnchor).isActive = true
        label.rightAnchor.constraint(equalTo: view.rightAnchor).isActive =
true

        let originalCenterY = label.center.y
        label.center.y += 400
        label.alpha = 1.0

        UIView.animate(withDuration: 2.5, delay: 0.5, usingSpringWithDamping:
0.8, initialSpringVelocity: 1.0, options: []) {
            label.center.y = originalCenterY
        }

    }
}

```



```

}

extension GameViewController: StoryboardInstantiable {
    static var storyboardInst: UIStoryboard {
        return UIStoryboard.main
    }
}

protocol GameViewControllerDelegate {
    func charCathed(gvc: GameViewController, charName: String)
}

```

Файл MapViewModel.swift

Реалізація функціональної задачі «Відображення доступних локацій»

```

import UIKit
import SVProgressHUD
import CoreLocation
import EasyAleert

class ShopsViewController: UIViewController {

    private var shops: [ShopModel] = [] {
        didSet {
            self.tableView.reloadData()
        }
    }

    private var locationManager = CLLocationManager()

    private var lastLocation: CLLocation?

    @IBOutlet weak var tableView: UITableView!

    override func viewDidLoad() {
        super.viewDidLoad()
        self.tableView.delegate = self
        self.tableView.dataSource = self

        self.locationManager.delegate = self
        self.locationManager.startUpdatingLocation()
    }
}

```

```

override func viewWillAppear(_ animated: Bool) {
    super.viewWillAppear(animated)
    SVProgressHUD.show()
    GetShopsRequest().request { shops, backendError in
        if let shops = shops {
            self.shops = shops.shops
        }
        SVProgressHUD.dismiss()
    }
}

@IBAction func onNavigate(_ sender: Any) {
    let mapVC = ShopMapViewViewController.create(shops: self.shops)
    self.navigationController?.pushViewController(mapVC, animated: true)
}

}

extension ShopsViewController: UITableViewDelegate {
    func tableView(_ tableView: UITableView, didSelectRowAt indexPath:
IndexPath) {
        let shop = self.shops[indexPath.item]
        if let uLocation = self.lastLocation {
            let distanceInMeters = uLocation.distance(from:
CLLocation(latitude: shop.lat, longitude: shop.lon))
            if (distanceInMeters < 500 || self.shops[indexPath.item].id == 2)
{
                let gvc = GameMapViewViewController.create(shop:
self.shops[indexPath.item])
                gvc.modalPresentationStyle = .fullScreen
                self.present(gvc, animated: true)
            } else {
                EasyAlert(delegate: self).showToast("Щоб розпочати квест
підійдіть ближче до магазину")
            }

        } else {
            EasyAlert(delegate: self).showToast("Неможливо визначити вашу
геолокацію")
        }
    }
}

```

```

}

extension ShopsViewController: UITableViewDataSource {
    func tableView(_ tableView: UITableView, numberOfRowsInSection section:
Int) -> Int {
        return self.shops.count
    }

    func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath)
-> UITableViewCell {
        let cell = tableView.dequeueReusableCell(withIdentifier:
"ShopTableViewCell") as! ShopTableViewCell
        cell.initialize(shop: self.shops[indexPath.item])
        return cell
    }

    func tableView(_ tableView: UITableView, heightForRowAt indexPath:
IndexPath) -> CGFloat {
        return 100
    }
}

extension ShopsViewController: CLLocationManagerDelegate {
    func locationManager(_ manager: CLLocationManager, didUpdateLocations
locations: [CLLocation]) {
        if let location = locations.last {
            self.lastLocation = location
        }
    }
}

extension ShopsViewController: EasyAlertDelegate {
    func show(_ alert: UIAlertController) {
        self.present(alert, animated: true, completion: nil)
    }
}

```

Файл Location.swift

Реалізація функціональної задачі «Відображення доступних локацій»

```

import UIKit
import GoogleMaps

```

```

import CoreLocation

class ShopMapsViewController: UIViewController {
    private var mapView: GMSMapView!

    private var locationManager = CLLocationManager()

    private var shops: [ShopModel] = []

    class func create(shops: [ShopModel]) -> ShopMapsViewController {
        let vc = ShopMapsViewController()
        ShopMapsViewController.storyboardInst.instantiateViewController(withIdentifier: ShopMapsViewController.identifier) as! ShopMapsViewController
        vc.shops = shops
        return vc
    }

    override func viewDidLoad() {
        super.viewDidLoad()

        self.initMapView()

        self.locationManager.delegate = self
        self.locationManager.startUpdatingLocation()
    }

    private func initMapView() {
        self.mapView = GMSMapView()
        self.reloadMapView()
        self.view.insertSubview(self.mapView, at: 0)

        self.mapView.isMyLocationEnabled = true

        self.shops.forEach { shop in
            let shopMarker = GMSMarker()
            shopMarker.icon = UIImage(named: "shopIcon")
            shopMarker.position = CLLocationCoordinate2D(latitude: shop.lat,
longitude: shop.lon)
            shopMarker.title = shop.name
            shopMarker.snippet = shop.name
            shopMarker.map = self.mapView
        }
    }
}

```

```

        override func viewWillTransition(to size: CGSize, with coordinator:
UIViewControllerTransitionCoordinator) {
            self.reloadMapView()
        }

        private func reloadMapView() {
            Timer.scheduledTimer(withTimeInterval: 0, repeats: false) { _ in
                let parentFrame = self.view.frame
                let navBarMaxY = self.navigationController?.navigationBar.frame.maxY ?? 0
                self.mapView.frame = CGRect(x: parentFrame.origin.x, y: navBarMaxY,
width:parentFrame.width, height: parentFrame.height - navBarMaxY)
            }
        }
    }

    extension ShopMapViewController: CLLocationManagerDelegate {
        func locationManager(_ manager: CLLocationManager, didUpdateLocations
locations: [CLLocation]) {
            if let location = locations.last {
                let camera = GMSCameraPosition.camera(withLatitude:
location.coordinate.latitude, longitude: location.coordinate.longitude, zoom:
17)

                self.mapView.animate(to: camera)
                self.locationManager.stopUpdatingLocation()
            }
        }
    }

    extension ShopMapViewController: StoryboardInstantiable {
        static var storyboardInst: UIStoryboard {
            return UIStoryboard.main
        }
    }
}

```

Файл KalmanFilter.swift

Реалізація функціональної задачі «Визначення точного місцязнаходження»

```
import Foundation
```

```

import CoreLocation

class KalmanFilter {
    var stateEstimate: CLLocationCoordinate2D
    var errorCovariance: Double

    let processNoise: Double
    let measurementNoise: Double

    init(initialEstimate: CLLocationCoordinate2D, initialErrorCovariance:
Double, processNoise: Double, measurementNoise: Double) {
        self.stateEstimate = initialEstimate
        self.errorCovariance = initialErrorCovariance
        self.processNoise = processNoise
        self.measurementNoise = measurementNoise
    }

    func update(measurement: CLLocationCoordinate2D) -> CLLocationCoordinate2D
{
        let kalmanGain = errorCovariance / (errorCovariance + measurementNoise)

        let newLatitude = stateEstimate.latitude + kalmanGain *
(measurement.latitude - stateEstimate.latitude)
        let newLongitude = stateEstimate.longitude + kalmanGain *
(measurement.longitude - stateEstimate.longitude)

        stateEstimate = CLLocationCoordinate2D(latitude: newLatitude,
longitude: newLongitude)

        errorCovariance = (1 - kalmanGain) * errorCovariance + processNoise

        return stateEstimate
    }
}

```

Файл LocationManager.swift

Реалізація функціональної задачі «Визначення точного місцязнаходження»

```

import Foundation
import CoreLocation
import Combine

```

```

class LocationManager: NSObject, ObservableObject, CLLocationManagerDelegate {
    private var locationManager = CLLocationManager()
    @Published var currentLocation: CLLocationCoordinate2D?

    private var kalmanFilter: KalmanFilter

    override init() {
        self.kalmanFilter = KalmanFilter(
            initialEstimate: CLLocationCoordinate2D(latitude: 0, longitude:
0),
            initialErrorCovariance: 1,
            processNoise: 1e-3,
            measurementNoise: 1e-2
        )
        super.init()
        locationManager.delegate = self
        locationManager.desiredAccuracy = kCLLocationAccuracyBest
        locationManager.requestWhenInUseAuthorization()
        locationManager.startUpdatingLocation()
    }

    func locationManager(_ manager: CLLocationManager, didUpdateLocations
locations: [CLLocation]) {
        guard let newLocation = locations.last else { return }

        let filteredLocation = kalmanFilter.update(measurement:
newLocation.coordinate)

        DispatchQueue.main.async {
            self.currentLocation = filteredLocation
        }
    }
}

```

Файл CurrentLocationView.swift

Реалізація функціональної задачі «Визначення точного місцязнаходження»

```

import SwiftUI
import MapKit

struct ContentView: View {

```

```

@StateObject private var locationManager = LocationManager()

var body: some View {
    VStack {
        if let currentLocation = locationManager.currentLocation {
            Map(coordinateRegion: .constant(MKCoordinateRegion(
                center: currentLocation,
                span: MKCoordinateSpan(latitudeDelta: 0.05,
longitudeDelta: 0.05)
            )))
            .frame(height: 300)
            Text("Current Location: \(currentLocation.latitude),
\ (currentLocation.longitude)")
        } else {
            Text("Locating...")
        }
    }
}

```

Файл ARObjectInteractionView.swift

Реалізація функціональної задачі «Взаємодії з віртуальними об'єктами»

```

import SwiftUI
import ARKit
import RealityKit
import Combine

struct ARObjectInteractionView: UIViewRepresentable {
    @Binding var objectCaught: Bool
    var userID: String
    var locationID: String

    func makeCoordinator() -> Coordinator {
        Coordinator(self, userID: userID, locationID: locationID)
    }

    func makeUIView(context: Context) -> ARView {
        let arView = ARView(frame: .zero)
        let config = ARWorldTrackingConfiguration()
        config.planeDetection = [.horizontal, .vertical]
        arView.session.run(config)
    }
}

```



```

        let tapGesture = UITapGestureRecognizer(target: context.coordinator,
action: #selector(Coordinator.handleTap(_:)))
        arView.addGestureRecognizer(tapGesture)

        return arView
    }

    func updateUIView(_ uiView: ARView, context: Context) {}

    class Coordinator: NSObject {
        var parent: ARObjInteractionView
        var arView: ARView { return parent.makeUIView(context: .init(parent:
parent)) }
        var userID: String
        var locationID: String

        init(_ parent: ARObjInteractionView, userID: String, locationID:
String) {
            self.parent = parent
            self.userID = userID
            self.locationID = locationID
        }

        @objc func handleTap(_ sender: UITapGestureRecognizer) {
            let location = sender.location(in: arView)
            let results = arView.hitTest(location, types:
[.existingPlaneUsingGeometry, .featurePoint])

            if let result = results.first {
                let entity = ModelEntity(mesh: .generateSphere(radius: 0.05))
                entity.position =
simd_make_float3(result.worldTransform.columns.3)

                let anchor = AnchorEntity(world: result.worldTransform)
                anchor.addChild(entity)

                arView.scene.addAnchor(anchor)
                parent.objectCaught = true

                let manager = RewardAndAchievementManager()
                manager.handleObjectCaught(for: userID, at: locationID)
            }
        }
    }

```

```

    }
  }
}

```

Файл ARInteractionView.swift

Реалізація функціональної задачі «Взаємодії з віртуальними об'єктами»

```

import SwiftUI

struct ARInteractionView: View {
    @State private var objectCaught = false
    var userID: String
    var locationID: String

    var body: some View {
        VStack {
            ARObjectInteractionView(objectCaught: $objectCaught, userID:
userID, locationID: locationID)
                .edgesIgnoringSafeArea(.all)
                .overlay(
                    Text(objectCaught ? "Object Caught!" : "Find and Tap the
Object")
                        .foregroundColor(.white)
                        .padding()
                        .background(Color.black.opacity(0.7))
                        .cornerRadius(10)
                        .padding(),
                    alignment: .top
                )
        }
    }
}

```

Файл RewardService.swift

Реалізація функціональної задачі «Отримання нагород та досягнень»

```

import Foundation

class RewardService {
    static let shared = RewardService()
    private let baseURL = URL(string: "https://hcr-
res.fra1.cdn.digitaloceanspaces.com/kpi/")!
}

```

```

func fetchRewards(for locationID: String, completion: @escaping
(Result<[Reward], Error>) -> Void) {
    let url = baseUrl.appendingPathComponent("rewards/\(locationID)")
    URLSession.shared.dataTask(with: url) { data, response, error in
        if let data = data {
            do {
                let rewards = try JSONDecoder().decode([Reward].self, from:
data)

                completion(.success(rewards))
            } catch {
                completion(.failure(error))
            }
        } else if let error = error {
            completion(.failure(error))
        }
    }.resume()
}

func addReward(to userID: String, reward: Reward, completion: @escaping
(Result<Bool, Error>) -> Void) {
    var request = URLRequest(url:
baseUrl.appendingPathComponent("users/\(userID)/rewards"))
    request.httpMethod = "POST"
    request.httpBody = try? JSONEncoder().encode(reward)
    request.setValue("application/json", forHTTPHeaderField: "Content-
Type")

    URLSession.shared.dataTask(with: request) { data, response, error in
        if let error = error {
            completion(.failure(error))
        } else {
            completion(.success(true))
        }
    }.resume()
}
}

```

Файл AchievementService.swift

Реалізація функціональної задачі «Отримання нагород та досягнень»

```
import Foundation
```

```

class AchievementService {
    static let shared = AchievementService()
    private let baseURL = URL(string: "https://hcr-
res.fra1.cdn.digitaloceanspaces.com/kpi/")!

    func fetchAchievements(for locationID: String, completion: @escaping
(Result<[Achievement], Error>) -> Void) {
        let url = baseURL.appendingPathComponent("achievements/\(locationID)")
        URLSession.shared.dataTask(with: url) { data, response, error in
            if let data = data {
                do {
                    let achievements = try
JSONDecoder().decode([Achievement].self, from: data)
                    completion(.success(achievements))
                } catch {
                    completion(.failure(error))
                }
            } else if let error = error {
                completion(.failure(error))
            }
        }.resume()
    }

    func addAchievement(to userID: String, achievement: Achievement,
completion: @escaping (Result<Bool, Error>) -> Void) {
        var request = URLRequest(url:
baseURL.appendingPathComponent("users/\(userID)/achievements"))
        request.httpMethod = "POST"
        request.httpBody = try? JSONEncoder().encode(achievement)
        request.setValue("application/json", forHTTPHeaderField: "Content-
Type")

        URLSession.shared.dataTask(with: request) { data, response, error in
            if let error = error {
                completion(.failure(error))
            } else {
                completion(.success(true))
            }
        }.resume()
    }
}

```

Файл RewardView.swift

Реалізація функціональної задачі «Перегляд досягнень та нагород»

```
import SwiftUI

struct RewardView: View {
    let reward: Reward

    var body: some View {
        VStack(alignment: .leading) {
            Text(reward.title)
                .font(.headline)
            Text(reward.description)
                .font(.subheadline)
            Image(uiImage: generateQRCode(from: reward.qrCode))
                .interpolation(.none)
                .resizable()
                .frame(width: 100, height: 100)
        }
        .padding()
        .background(Color.white)
        .cornerRadius(10)
        .shadow(radius: 5)
    }

    private func generateQRCode(from string: String) -> UIImage {
        let data = string.data(using: String.Encoding.ascii)
        if let filter = CIFilter(name: "CIQRCodeGenerator") {
            filter.setValue(data, forKey: "inputMessage")
            let transform = CGAffineTransform(scaleX: 10, y: 10)
            if let output = filter.outputImage?.transformed(by: transform) {
                return UIImage(ciImage: output)
            }
        }
        return UIImage(systemName: "xmark.circle") ?? UIImage()
    }
}
```

Файл QRCodeGenerator.swift

Реалізація функціональної задачі «Застосування нагороди в рамках програми лояльності»

```
import UIKit
import CoreImage.CIFilterBuiltins

class QRCodeGenerator {
    static func generateQRCode(from string: String) -> UIImage? {
        let data = Data(string.utf8)
        let filter = CIFilter.qrCodeGenerator()
        filter.setValue(data, forKey: "inputMessage")

        guard let outputImage = filter.outputImage else { return nil }
        let transform = CGAffineTransform(scaleX: 10, y: 10)
        let scaledImage = outputImage.transformed(by: transform)

        guard let cgImage = CIContext().createCGImage(scaledImage, from:
scaledImage.extent) else { return nil }
        return UIImage(cgImage: cgImage)
    }
}
```

Файл RewardDetailView.swift

Реалізація функціональної задачі «Застосування нагороди в рамках програми лояльності»

```
import SwiftUI

struct RewardDetailView: View {
    let reward: Reward

    var body: some View {
        VStack {
            Text(reward.title)
                .font(.largeTitle)
                .padding(.bottom, 20)

            if let qrCodeImage = QRCodeGenerator.generateQRCode(from:
reward.qrCode) {
                Image(uiImage: qrCodeImage)
                    .interpolation(.none)
                    .resizable()
            }
        }
    }
}
```

```
        .frame(width: 200, height: 200)
        .padding()
    }

    Text(reward.description)
        .font(.body)
        .padding()

    Spacer()
}
.padding()
.navigationTitle("Reward Details")
}
}
```

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ
РЕАЛЬНОСТІ**

Програма та методика тестування

КП.ІП-0104.045490.04.51

“ПОГОДЖЕНО”

Керівник роботи:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав БЕРЛІНСЬКИЙ

Київ – 2024

ЗМІСТ

1	ОБ’ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	5
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробувань є мобільний додаток для геолокаційної гри з використанням розширеної реальності. Цей додаток створений для платформи iOS.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- перевірка збереження даних;
- знаходження проблем, помилок і недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування - перевіряється програма разом з усією документацією, яка аналізується на предмет дотримання стандартів програмування;
- динамічне тестування - застосовується в процесі виконання програми. Коректність програмного засобу перевіряється на певній кількості тестів. При прогоні кожного з них збираються та аналізуються дані про проблеми та помилки в роботі програми;
- мануальне тестування – тестування без використання автоматизації, тест-кейси пише особа, що тестує програмне забезпечення;

Було обрано саме такі варіанти тестування з метою забезпечення комплексного підходу до перевірки якості та надійності мобільного додатку для геолокаційної гри з використанням розширеної реальності. Статичне тестування дозволяє виявити можливі проблеми на ранніх етапах розробки шляхом перевірки коду та документації. Це забезпечує дотримання стандартів програмування і виявлення потенційних помилок, які можуть виникнути в процесі кодування, ще до запуску програми. Динамічне тестування застосовується в процесі виконання програми, що дозволяє перевірити коректність роботи додатку в реальних умовах. Прогін тестів на різних сценаріях використання дозволяє збирати та аналізувати дані про проблеми та помилки, що виникають під час роботи програми. Це допомагає виявити баги, які не були помічені під час статичного тестування. Мануальне тестування передбачає перевірку функціональності додатку без використання автоматизації, що дозволяє тестувальникам детально перевірити всі аспекти роботи програми з точки зору кінцевого користувача. Тест-кейси, написані вручну, дозволяють гнучко підходити до тестування специфічних сценаріїв використання та виявляти помилки, які можуть залишитися непоміченими під час автоматизованого тестування.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Для того, щоб перевірити працездатність та відмовостійкість застосунку, необхідно провести наступні тестування:

- динамічне тестування на відповідність функціональним вимогам;
- тестування на мобільних пристроях з різною роздільною здатністю екрану;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування зміни орієнтації екрану;
- тестування працездатності програми у випадку відсутності з'єднання до мережі;
- тестування інтерфейсу користувача;

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ
РЕАЛЬНОСТІ**

Керівництво користувача

КП.ІП-0104.045490.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Ярослав БЕРЛІНСЬКИЙ

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ.....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	4
2.3	Перевірка коректної роботи.....	4
3	ВИКОНАННЯ ПРОГРАМИ	5

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

Мобільний додаток для геолокаційної гри з використанням розширеної реальності призначений для інтерактивного залучення користувачів шляхом інтеграції віртуальних об'єктів у реальний світ. Додаток дозволяє користувачам взаємодіяти з віртуальними об'єктами, отримувати нагороди та використовувати їх у реальних закладах, що є частиною програми лояльності. Основною метою додатку є надання користувачам захоплюючого ігрового досвіду, який стимулює відвідування партнерських закладів, покращуючи взаємодію між клієнтами та бізнесами.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Для успішної роботи мобільного додатку для геолокаційної гри з використанням розширеної реальності необхідно дотримуватися певних системних вимог:

- наявність мобільного пристрою з операційною системою iOS 17.0 або вище.
- наявність стабільного доступу до Інтернету зі швидкістю більше 10 мегабіт/с.
- наявність принаймні 1 гб вільного місця на мобільному пристрої.
- достатній рівень заряду батареї або доступ до джерела живлення для уникнення перебоїв у роботі додатку через високий рівень енергоспоживання.

2.2 Завантаження застосунку

Додаток можна завантажити з платформи AppStore, натиснувши на кнопку інсталяції на iOS-смартфон.

2.3 Перевірка коректної роботи

У разі успішної інсталяції застосунку, його іконка з'явиться на робочому столі смартфона, після чого відбудеться запуск додатку.

3 ВИКОНАННЯ ПРОГРАМИ

При першому запуску програмного забезпечення користувач бачить екран реєстрації (рис. 3.1) або, якщо він вже зареєстрований, екран авторизації (рис. 3.2). У випадку реєстрації, користувачеві потрібно ввести своє ім'я, електронну пошту та придумати пароль. Якщо ж він збирається авторизуватися, то потрібно буде ввести свою електронну пошту та пароль. Користувач також може повернутися на екран зміни пароля, якщо його забув.

Якщо користувач бажає авторизуватися під іменем адміністратора, то він має після введення своїх даних натиснути на відповідну кнопку входу адміністратора (рис. 3.2).

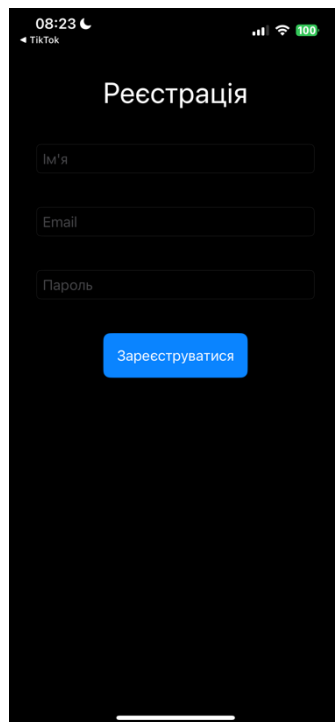


Рисунок 3.1 – Екран реєстрації

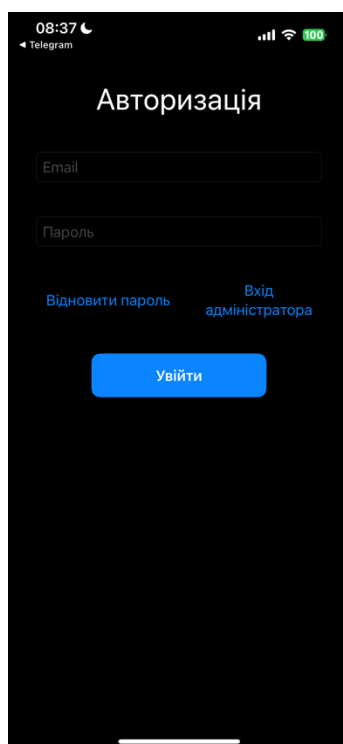


Рисунок 3.2 – Екран авторизації

Користувач потрапляє в головне меню завдань, де він бачить перелік місць, де можна взяти участь в програмі лояльності з цим застосунком (рис. 3.3)

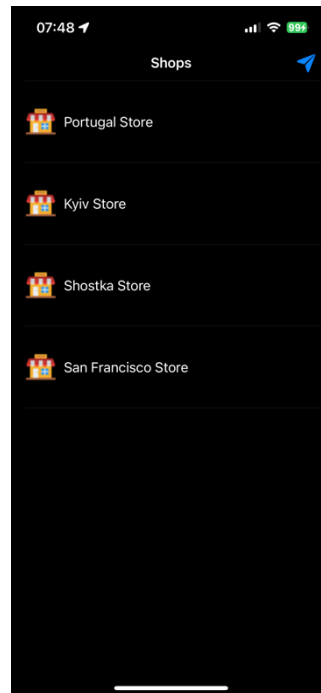


Рисунок 3.3 – Екран зі списком локацій, де діють програми лояльності

Також користувач має змогу подивитися на місця, які знаходяться в безпосередній близькості, за допомогою карти, натиснувши на стрілочку в правому верхньому кутку та надавши доступ на визначення місця знаходження (рис. 3.4) та перейшовши на екран самої мапи (рис. 3.5), де можна вибрати доступний магазин з програмою лояльності та взяти участь в квесті (але при умові, що користувач знаходиться близько).

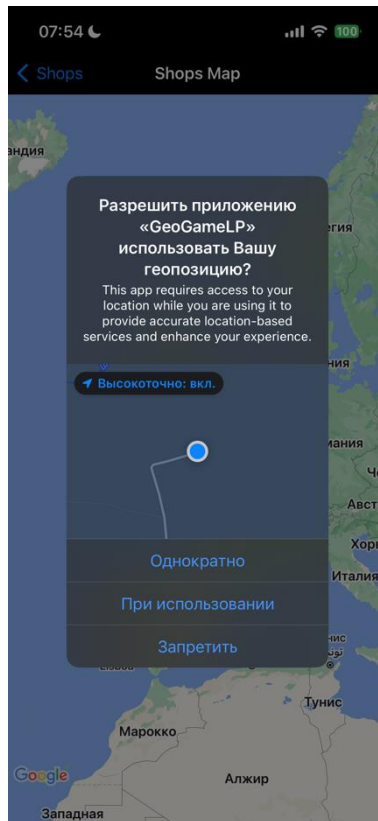


Рисунок 3.4 – Надання доступу до геоданих

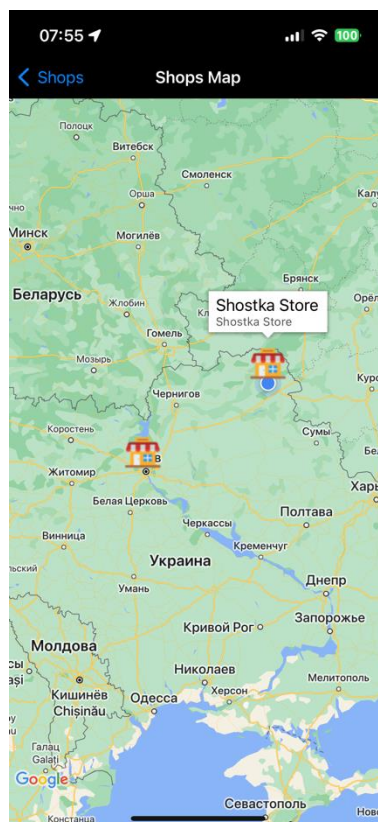


Рисунок 3.5 – Экран перегляду локацій з програмами лояльності

Обрати локацію з програмою лояльності можна також і повернувшись в головне меню із завданнями, де користувач отримає або сповіщення про заборону через недостатню близькість до місця проведення програми лояльності (рис. 3.6), або ж мати змогу взяти участь у квесті (рис. 3.7)

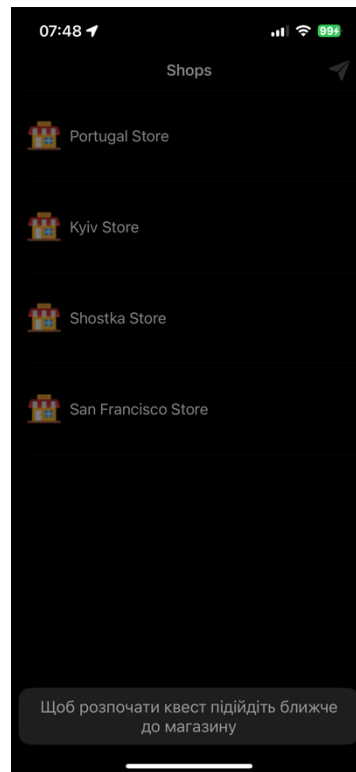


Рисунок 3.6 – Сповіщення про заборону прийняти участь у програмі лояльності

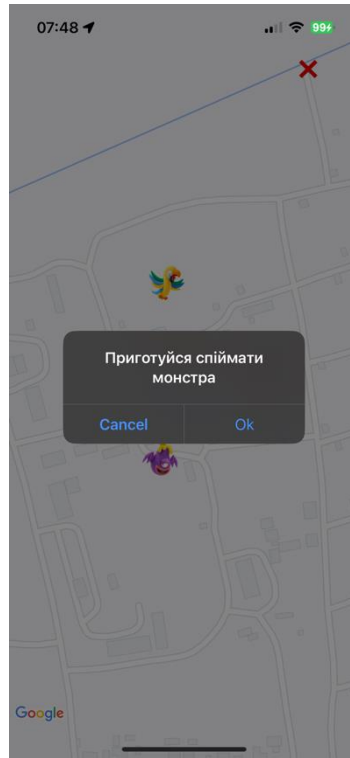


Рисунок 3.8 – Початок квесту програми лояльності

На екрані квесту користувач бачить істот або предмети на карті, які треба спіймати, щоби виконати квест. Якщо користувач перейде до місця знаходження зображуваного віртуального об'єкту, то він матиме змогу його спіймати через камеру свого смартфона (рис. 3.9), але спочатку об'єкт потрібно відшукати камерою, повертаючи її в різні сторони.



Рисунок 3.9 – Екран взаємодії з віртуальним об'єктом

Взаємодіяти з об'єктом можна завдяки жестам по екрану. Спіймати об'єкт можна шляхом натискання по ньому (рис. 4.12)



Рисунок 3.10 – Спійманий об'єкт

Якщо після успішного виконання квесту або квестів користувач отримує нагороду, то він може використати її в рамках програми лояльності через QR-код (рис. 3.11).



Рисунок 3.11 – Екран використання нагороди

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**ГЕОЛОКАЦІЙНА ГРА З ВИКОРИСТАННЯМ РОЗШИРЕНОЇ
РЕАЛЬНОСТІ**

Графічний матеріал

КПІ.ПІ-0104.045490.06.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Микола ХРАМЧЕНКО

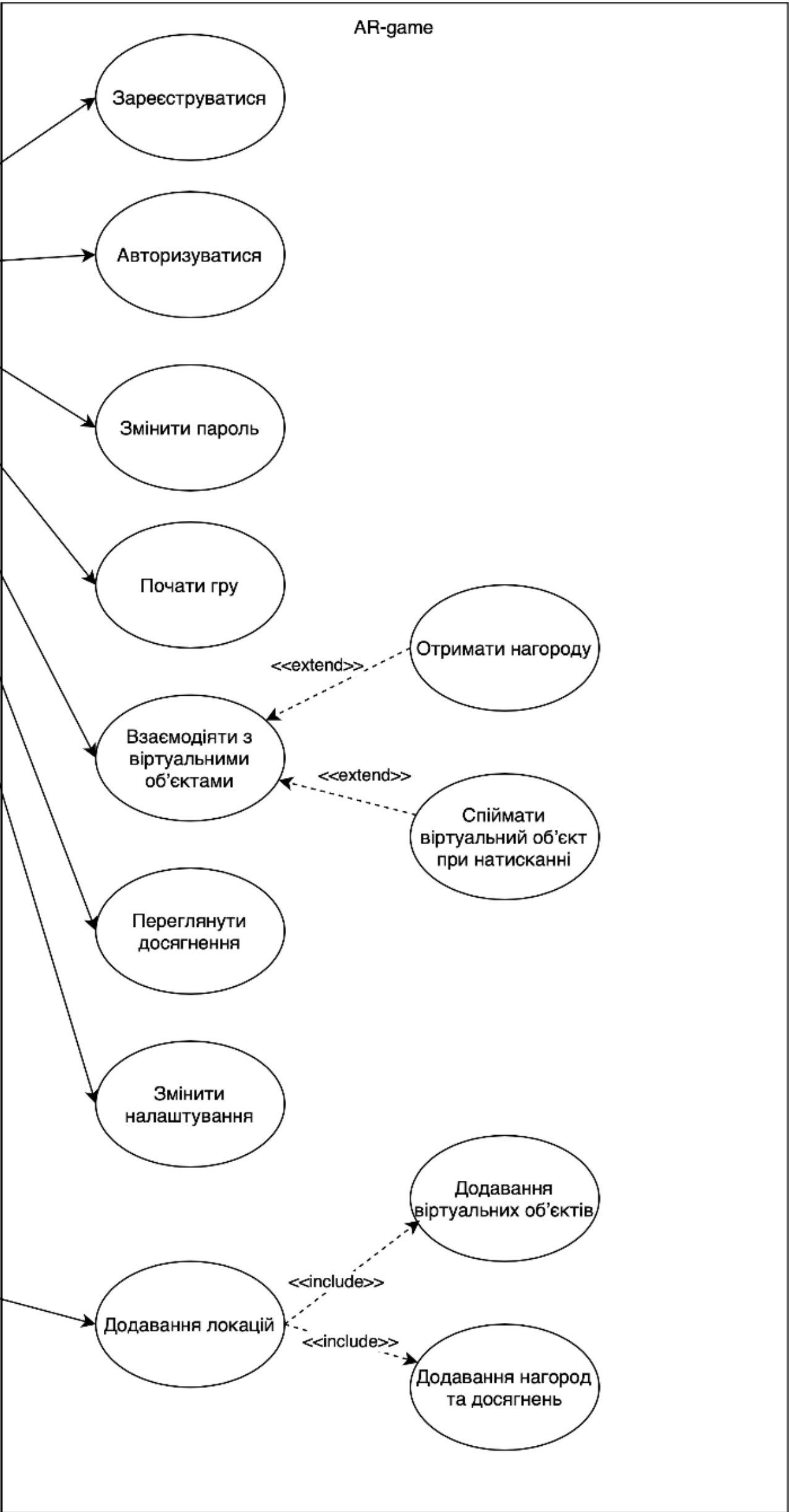
Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

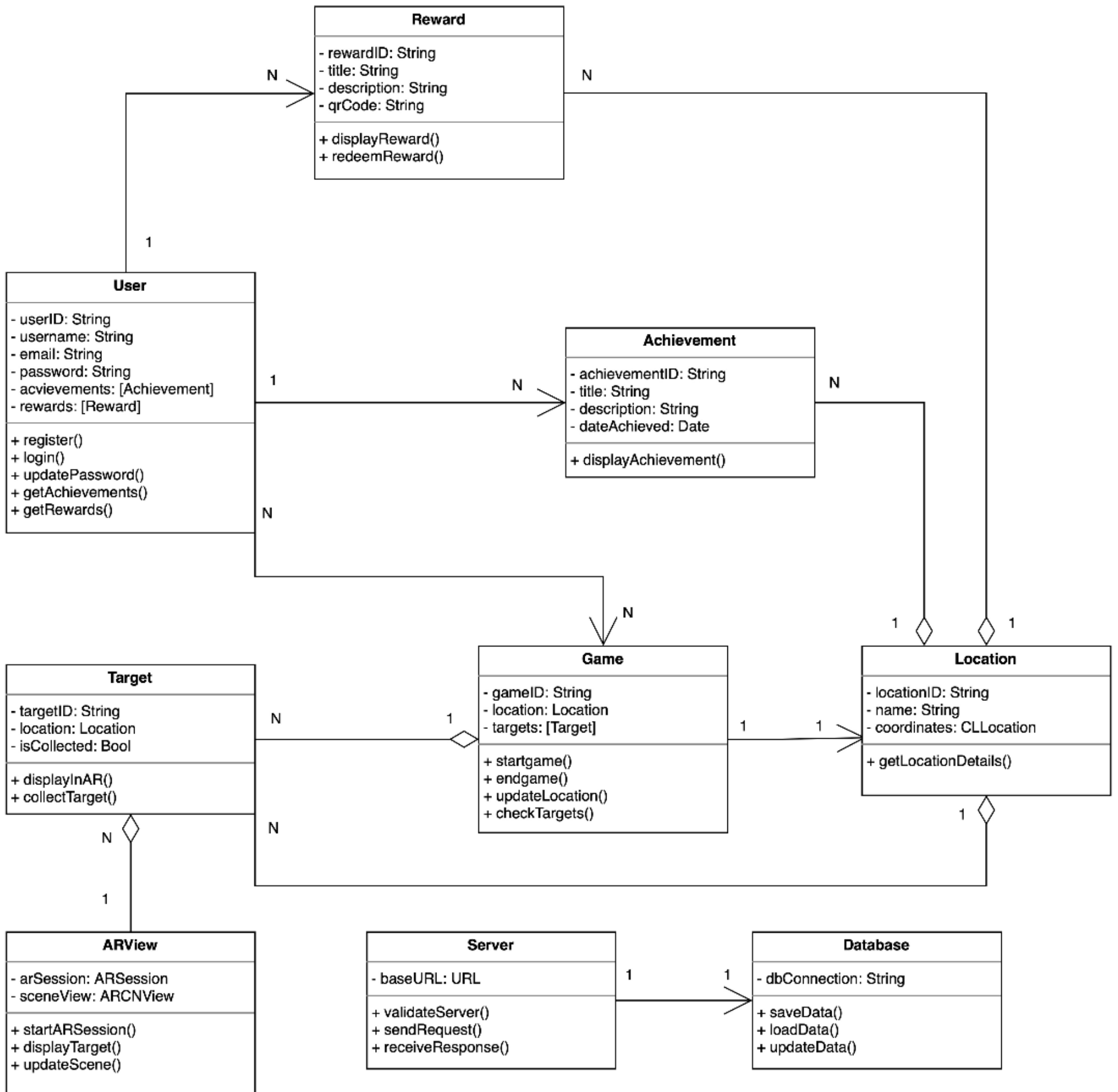
Виконавець:

_____ Ярослав БЕРЛІНСЬКИЙ

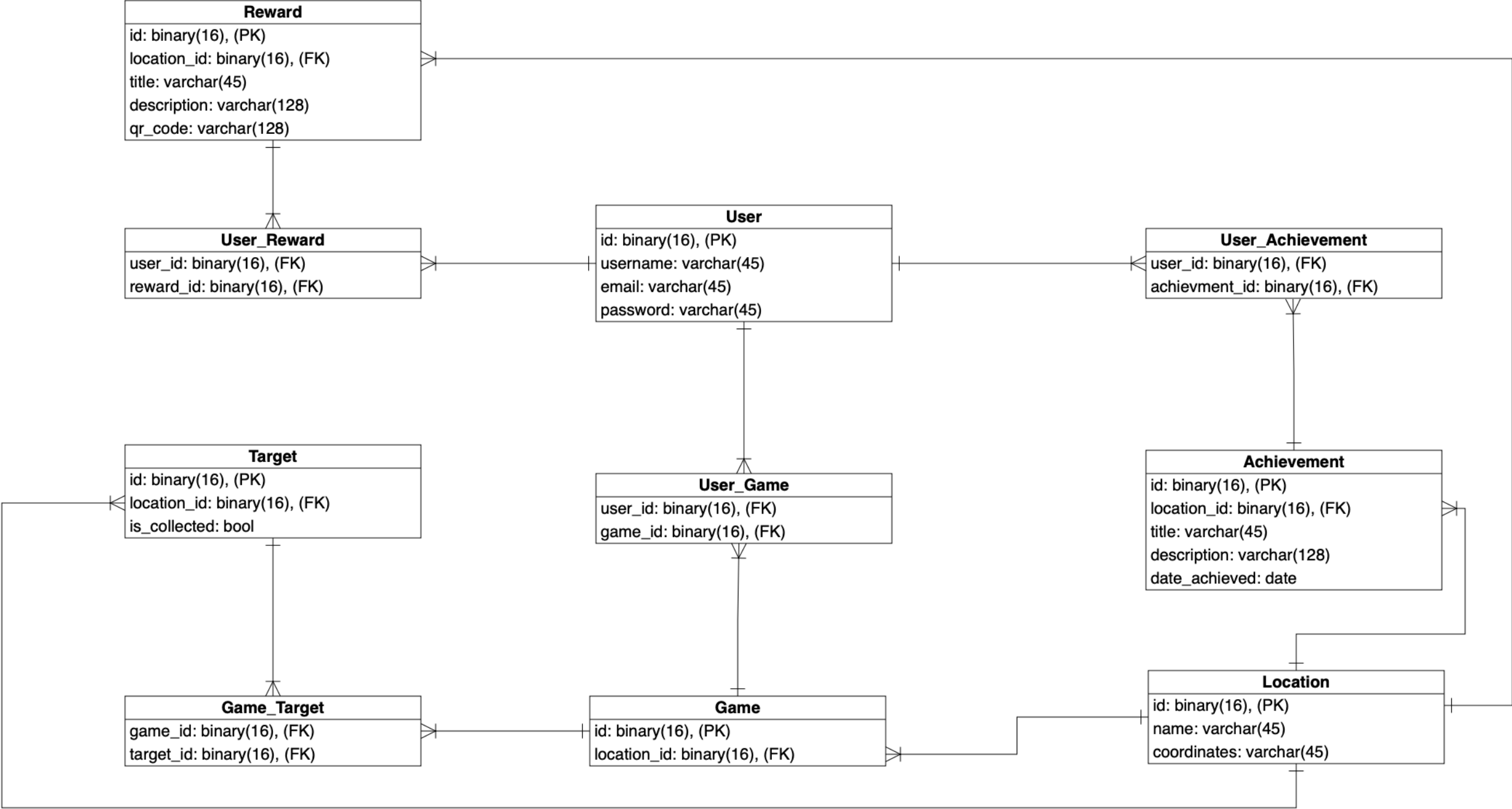
Київ – 2024



						КПІ.ІП-0104.045490.06.99.ССВ									
						Схема структурна варіантів використань					Лит.		Маса	Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата											
Розробив	Берлінський Я.В.														
Перевірів	Храмченко М.С.														
Т. контр.											1		1		
Н. контр.		Головченко М.М.				Геолокаційна гра з використанням розширеної реальності					КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01				
Затвердив		Жаріков Е.В.													



					КПІ.ІП-0104.045490.06.99.CCK			
Зм.	Арк.	№ докум.	Підп.	Дата	Схема структурна класів програмного забезпечення	Лит.	Маса	Масштаб
Розробив		Берлінський Я.В.						
Перевішив		Храмченко М.С.						
Т. контр.						1		1
Н. контр.		Головченко М.М.			Геолокаційна гра з використанням розширеної реальності	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01		
Затвердив		Жаріков Е.В.						



					КПІ.ІП-0104.045490.06.99.СБД				
					Схема бази даних	Лит.	Маса	Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата					
Розробив		Берлінський Я.В.							
Перевішив		Храмченко М.С.							
Т. контр.						1	1		
Н. контр.		Головченко М.М.			Геолокаційна гра з використанням розширеної реальності	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01			
Затвердив		Жаріков Е.В.							